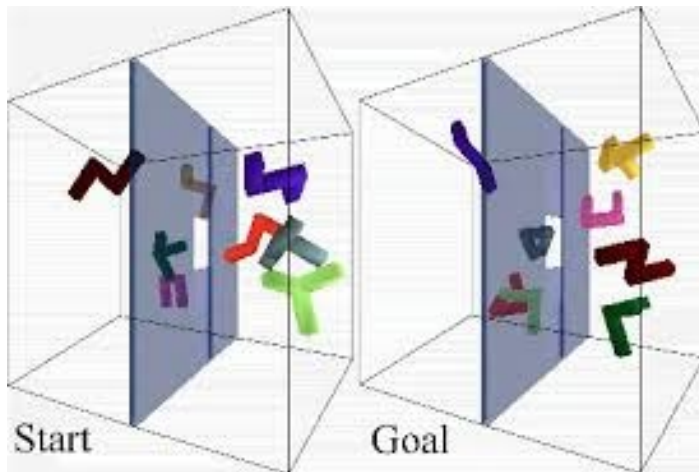
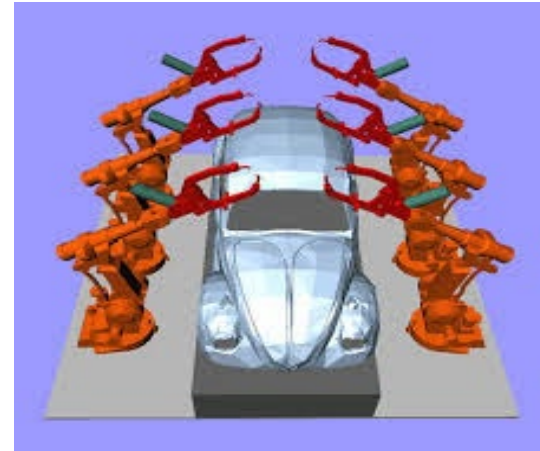


Motion Planning (Master Computer Science)



Organisation

Lecturer: Thomas Horsch, thomas.horsch@h-da.de
Tel. 533-68457, Office: Building D14 Room 1.08

Assistant: Rudi Scheitler, rudi.scheitler@h-da.de
Tel. 533-68465, Office: Building D10 Room 0.36

Ressources:

Choset and others: Principles of Robot Motion, MIT Press

Steven M. LaValle: Planning Algorithms (english, PDF eBook, Link in module description)

David Hsu, National University of Singapore

Lynch, Park: Modern Robotics:

http://hades.mech.northwestern.edu/index.php/Modern_Robotics

Software:

CoppeliaSim: Simulation Platform

OMPL: The Open Motion Planning Library (Kavraki)

Organisation

■ Exam:

- date will be announced (see OBS)
- form of exam depends on number of students and will be announced at beginning of the course
- approval: successful practical exercises (100%)

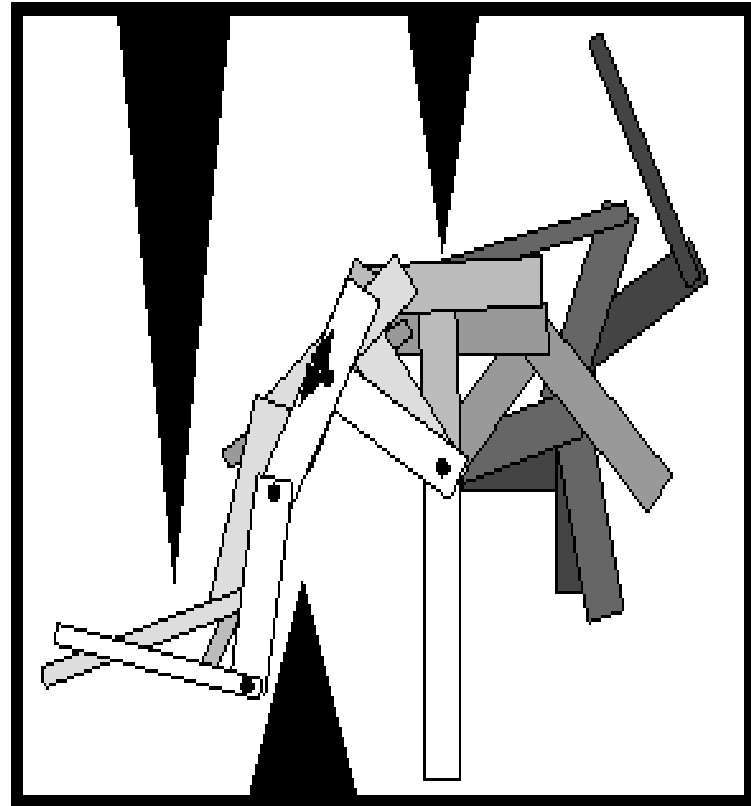
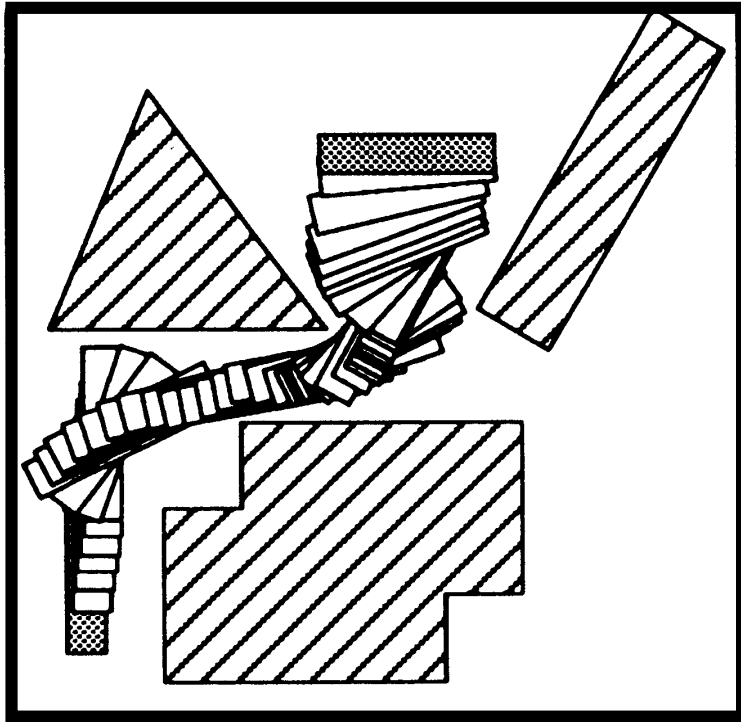
Execution of practical exercise:

- exercises - pairwise
- presence mandatory
- Delivery via Moodle (subscription key: see OBS)

Expectations of knowledge

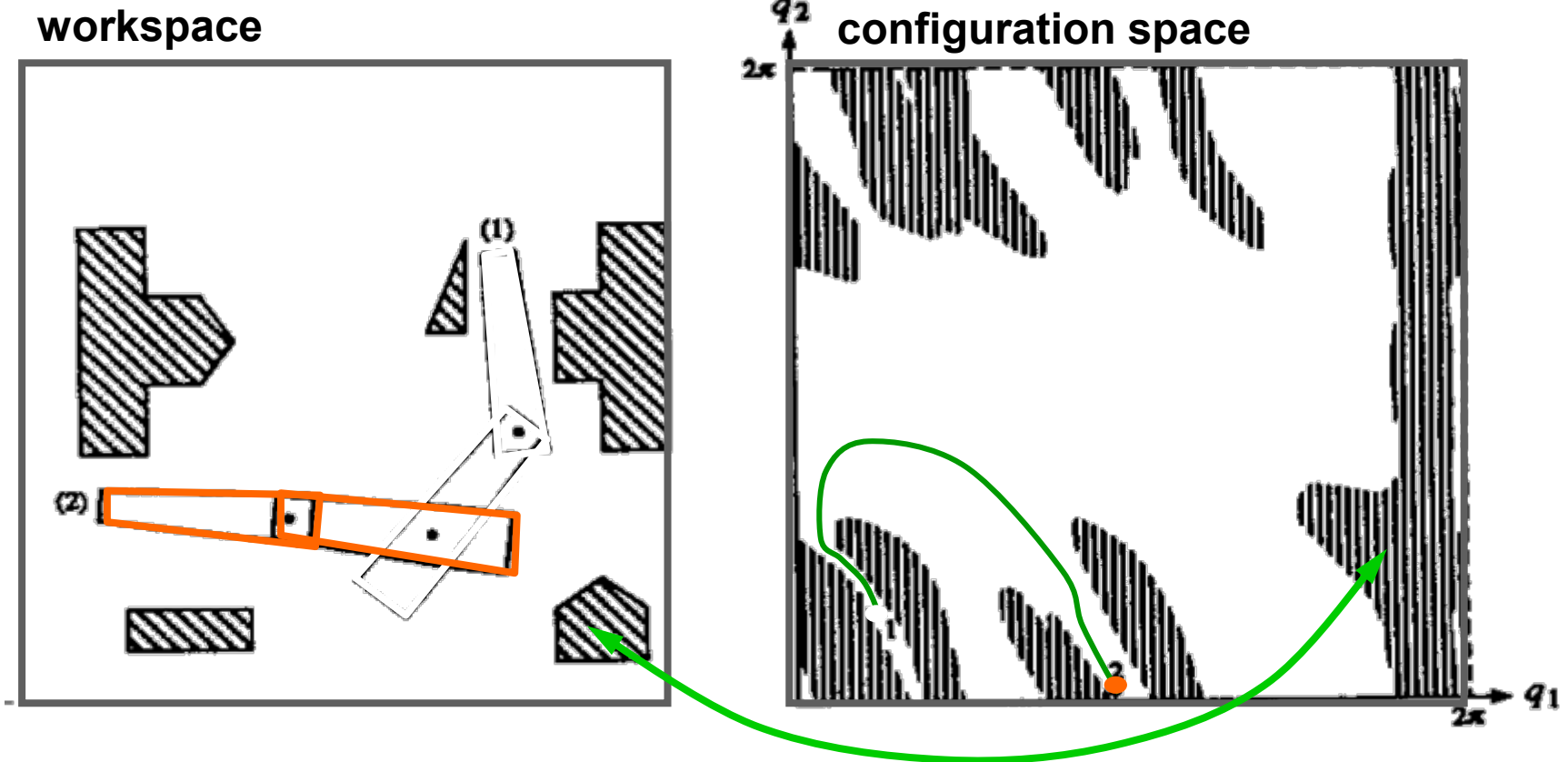
- Linear algebra, probability theory (bachelor level)
- Programming

What is a path?

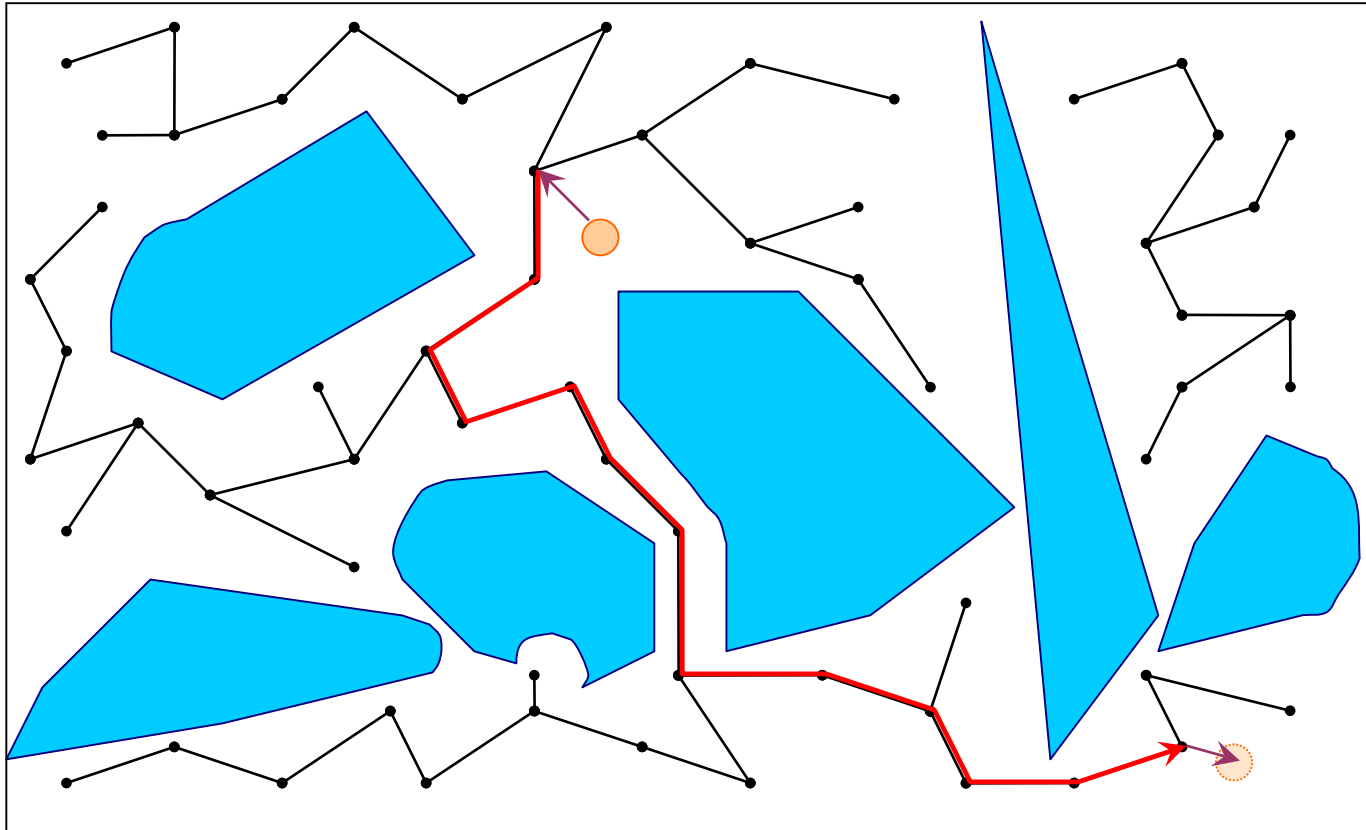


Rough idea of configuration space (c-space)

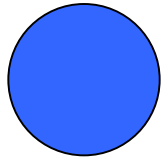
- Convert rigid robots, articulated robots, *etc.* into points
- Apply algorithms for moving points
- Mapping from the workspace to the configuration space



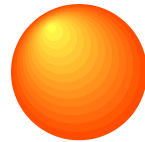
What is a Configuration Space ?



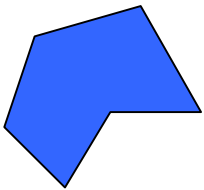
Degrees of Freedom



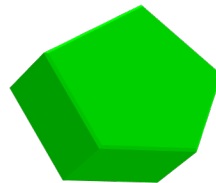
Disc robot:
2 dofs: (x, y)



Ball robot (flyer):
3 dofs: (x, y, z)



Polygonal robot:
3 dofs: (x, y, θ)



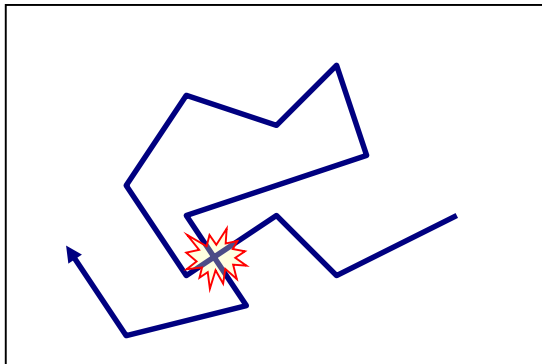
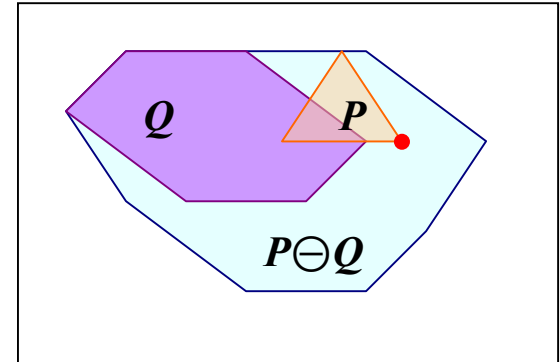
Polyhedral robot (flyer):
6 dofs: $(x, y, z, \theta_x, \theta_y, \theta_z)$



Poly-line robot:
 $n+2$ dofs: $(x, y, \theta_1, \theta_2, \dots, \theta_n)$

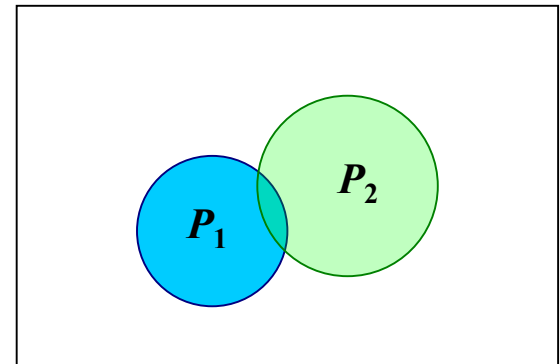
Forbidden Configurations

Due to robot–obstacle collisions
(configuration within a C-obstacle)

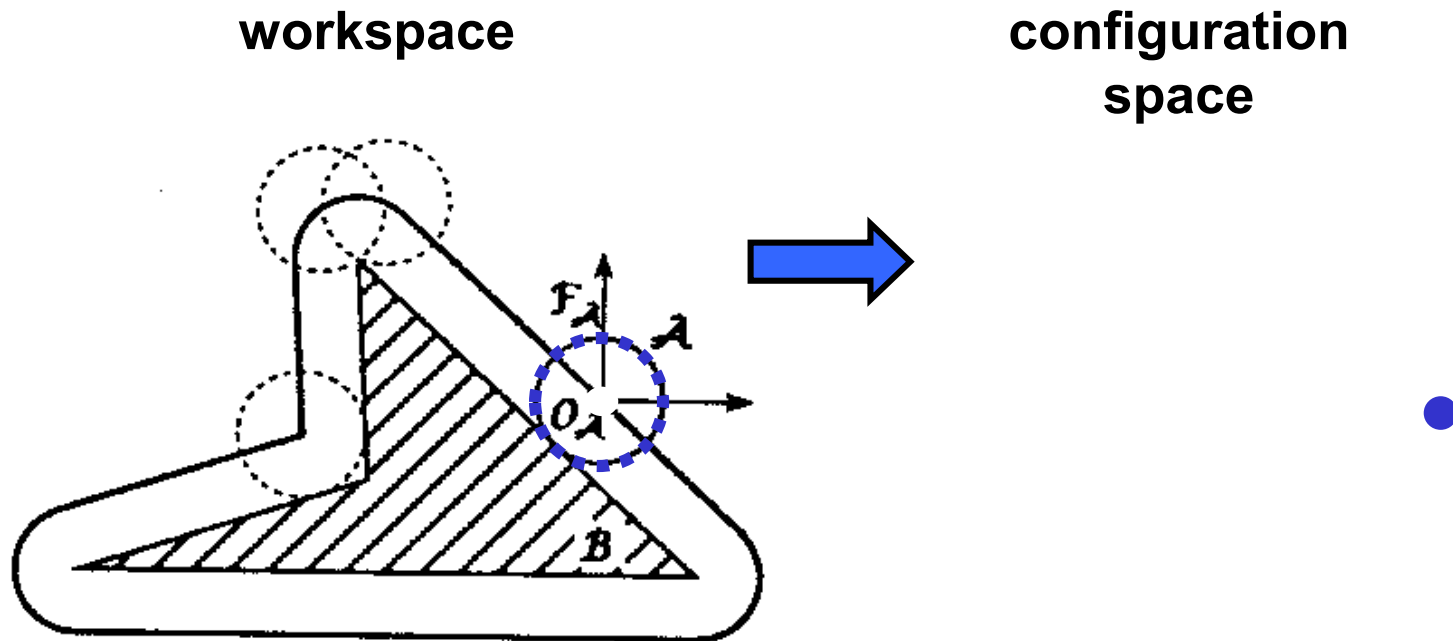


Due to self-collisions

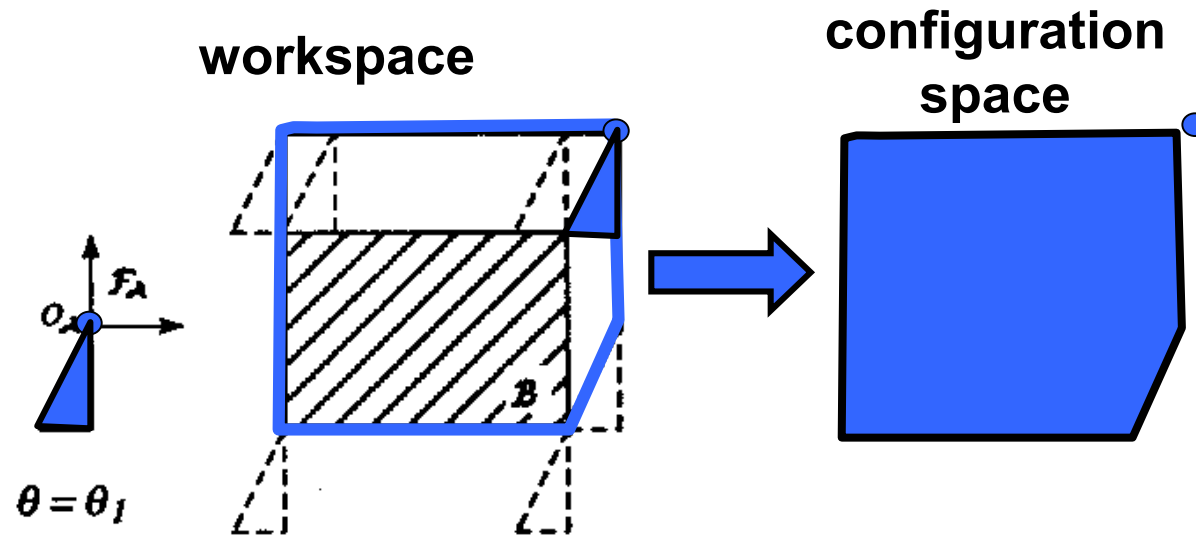
Due to robot–robot collisions
(in case of multiple robots)



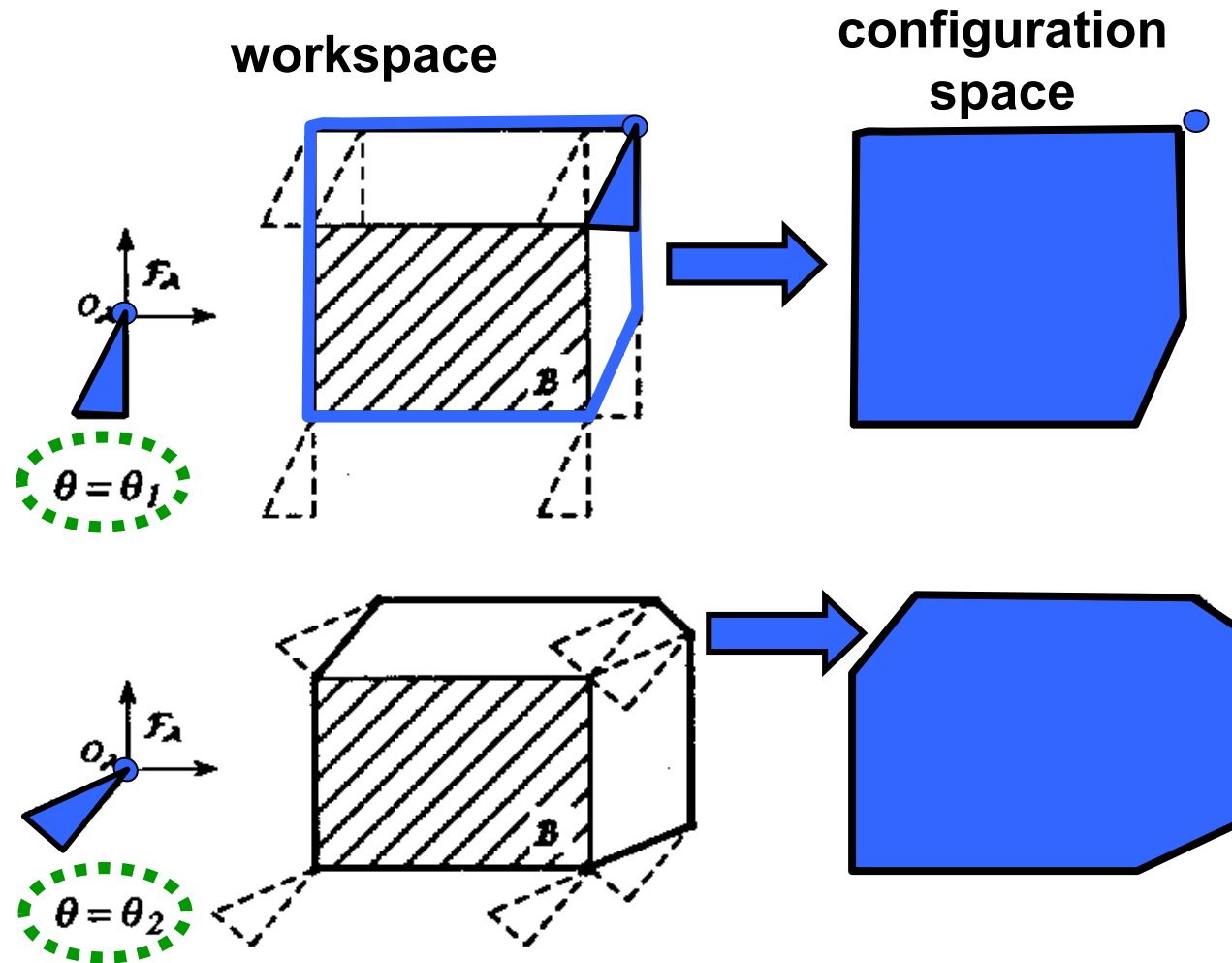
Disc in 2-D workspace



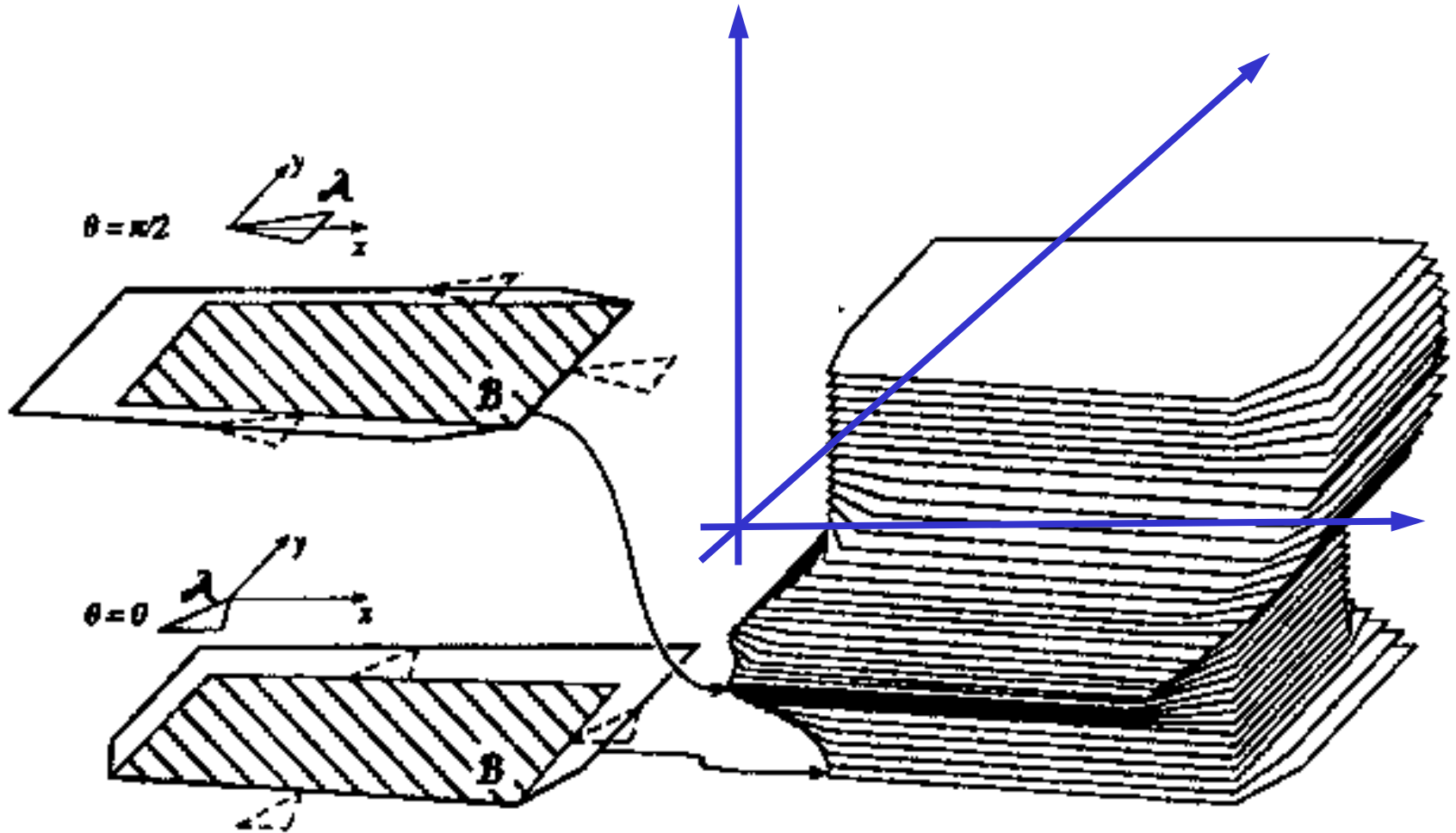
Polygonal robot translating in 2-D workspace



Polygonal robot translating & rotating in 2-D workspace



Polygonal robot translating & rotating in 2-D workspace



Constraints on paths

A **trajectory** is a path parameterized by time:

$$\tau : t \in [0, T] \rightarrow \tau(t) \in C$$

Constraints

- Finite length
- Bounded curvature
- Smoothness
- Minimum length
- Minimum time
- Minimum energy
- ...

Free space topology

- A **free** path lies entirely in the free space F .
- The moving object and the obstacles are modeled as closed subsets, meaning that they contain their boundaries.
- One can show that the C-obstacles are closed subsets of the configuration space C as well.
- Consequently, the free space F is an open subset of C . Hence, each free configuration is the center of a ball of non-zero radius entirely contained in F .

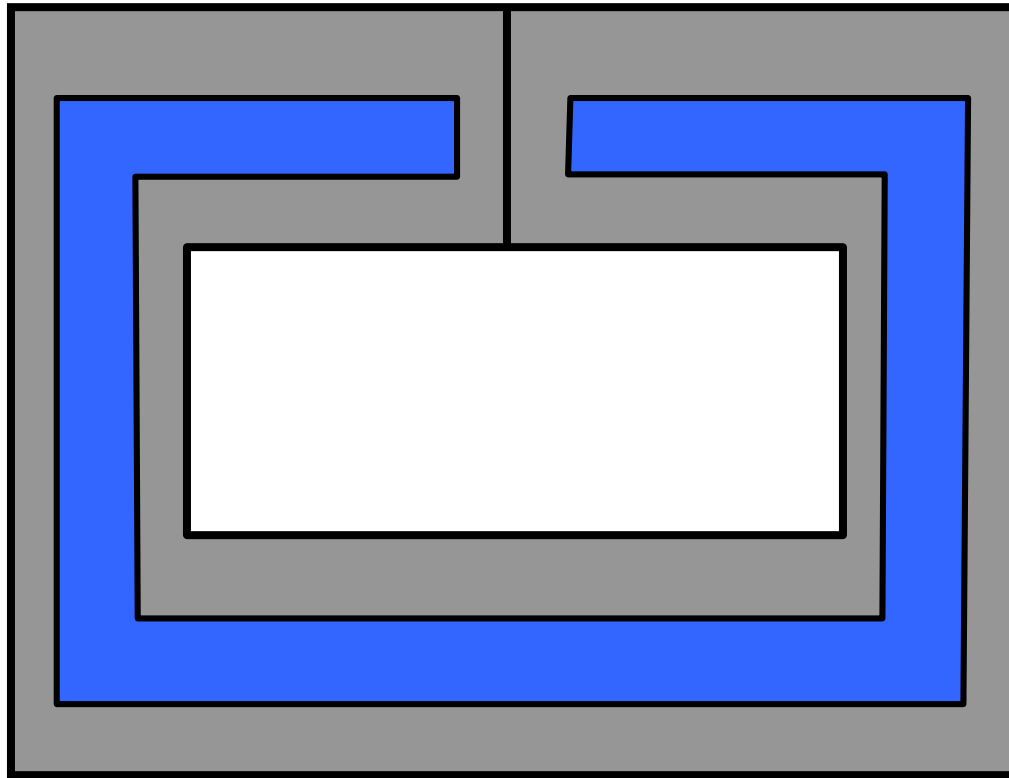
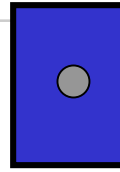
Semi-free space

A configuration q is **semi-free** if the moving object placed at q touches the boundary, but not the interior of obstacles.

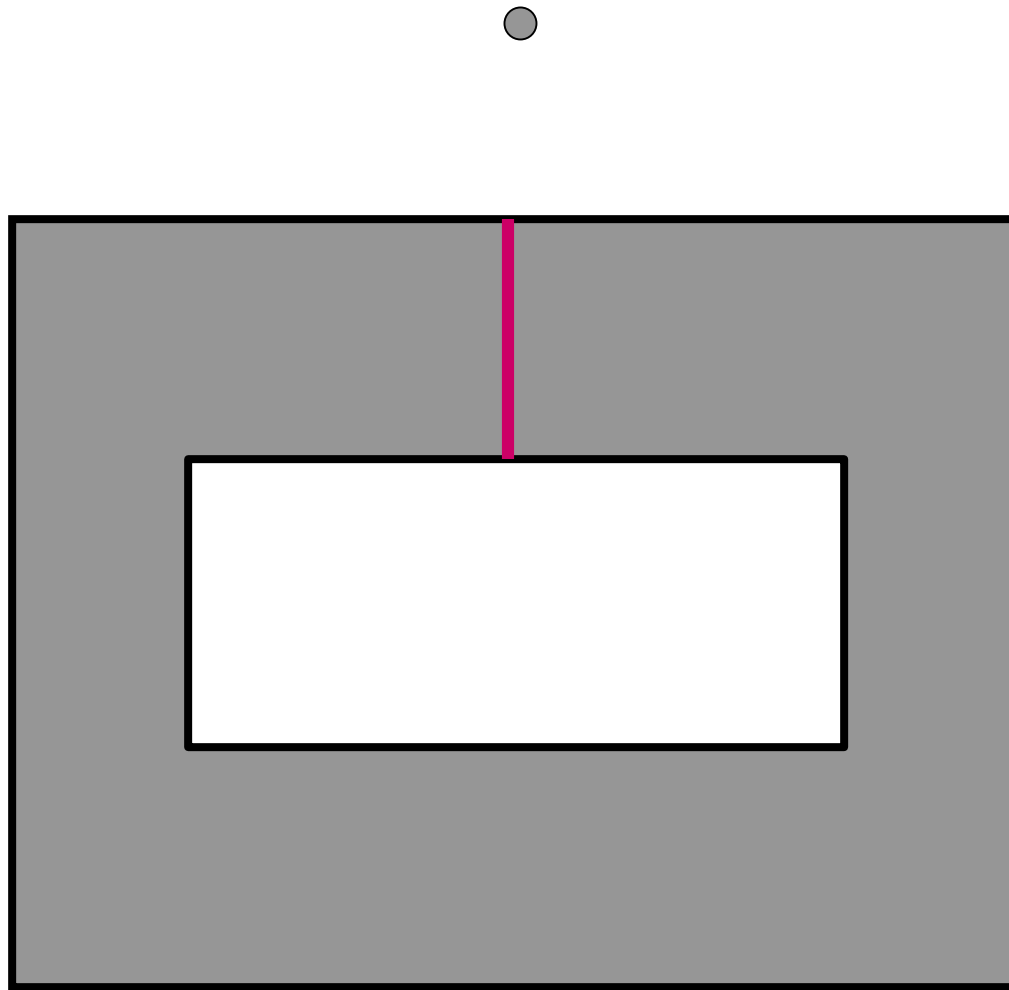
- Free, or
- in contact

The semi-free space is a closed subset of C . Its boundary is a superset of the boundary of F .

Example



Example



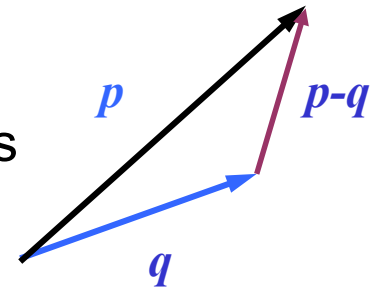
Minkowski sum

- The **Minkowski sum** of two sets P and Q , denoted by $P \oplus Q$, is defined as

$$P \oplus Q = \{ p+q \mid p \in P, q \in Q \}$$

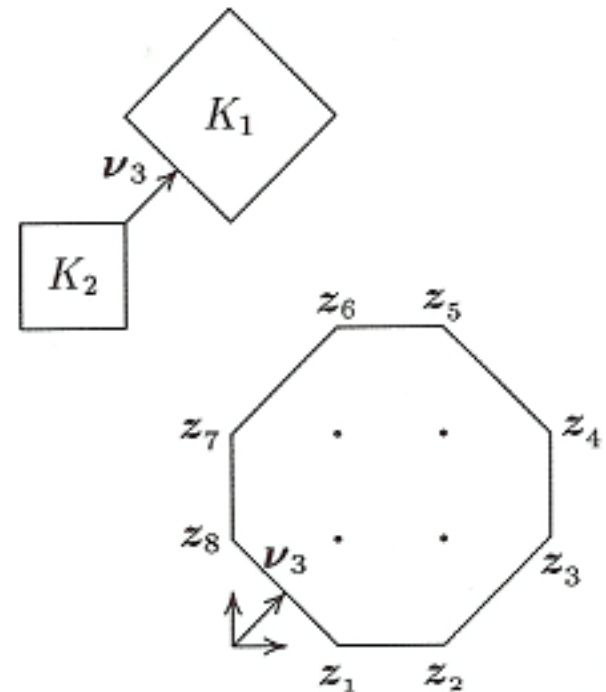
- Similarly, the **Minkowski difference** is defined as

$$P \ominus Q = \{ p-q \mid p \in P, q \in Q \}$$



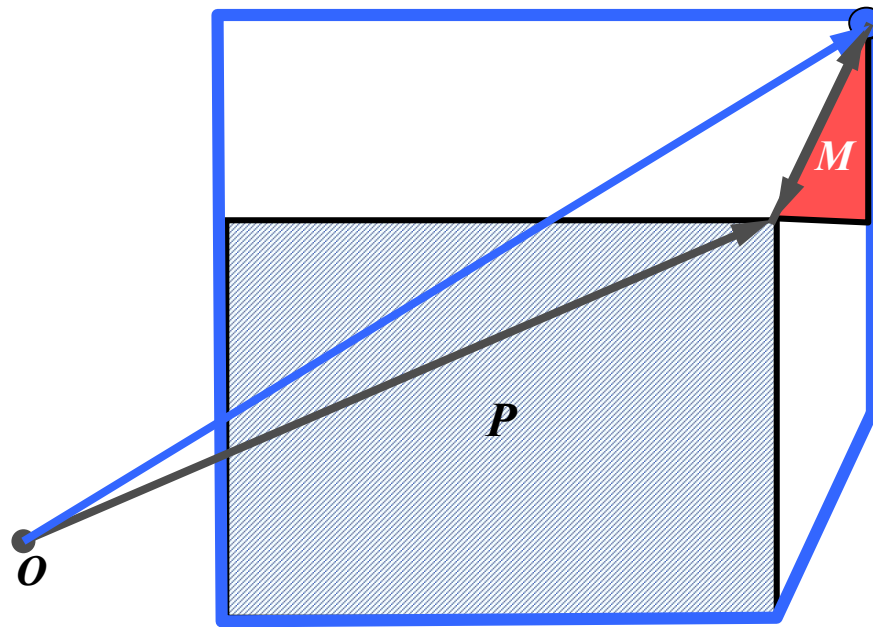
Minkowski sum of convex polygons

- The Minkowski sum of two convex polygons P and Q of m and n vertices respectively is a convex polygon $P \oplus Q$ of $m + n$ vertices.
 - The vertices of $P \oplus Q$ are the “sums” of vertices of P and Q .
- Example: Minkowski difference



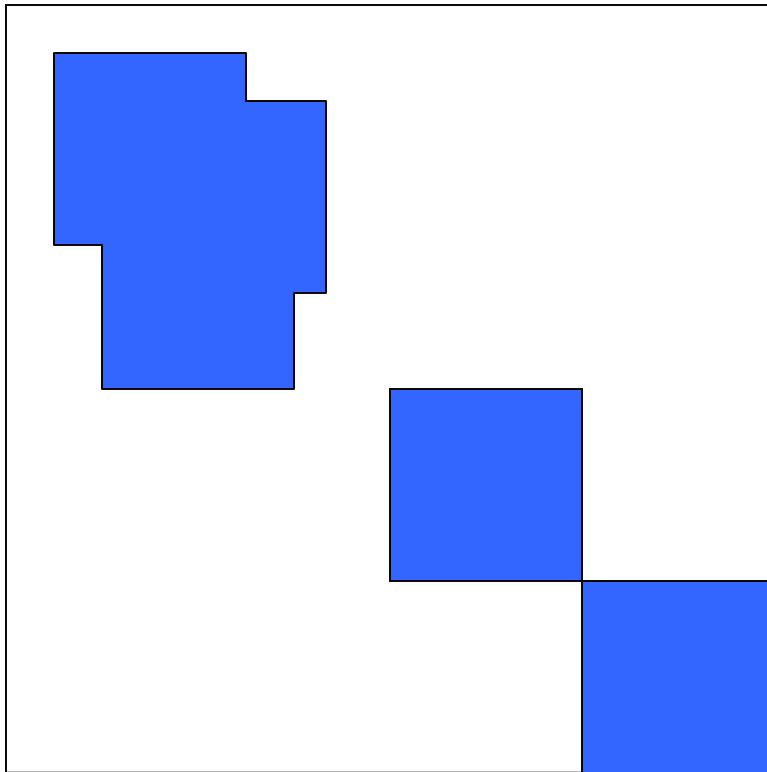
Observation

- If P is an obstacle in the workspace and M is a moving object. Then the C-space obstacle corresponding to P is $P \ominus M(0)$.



Quadtree Example

Space Representation

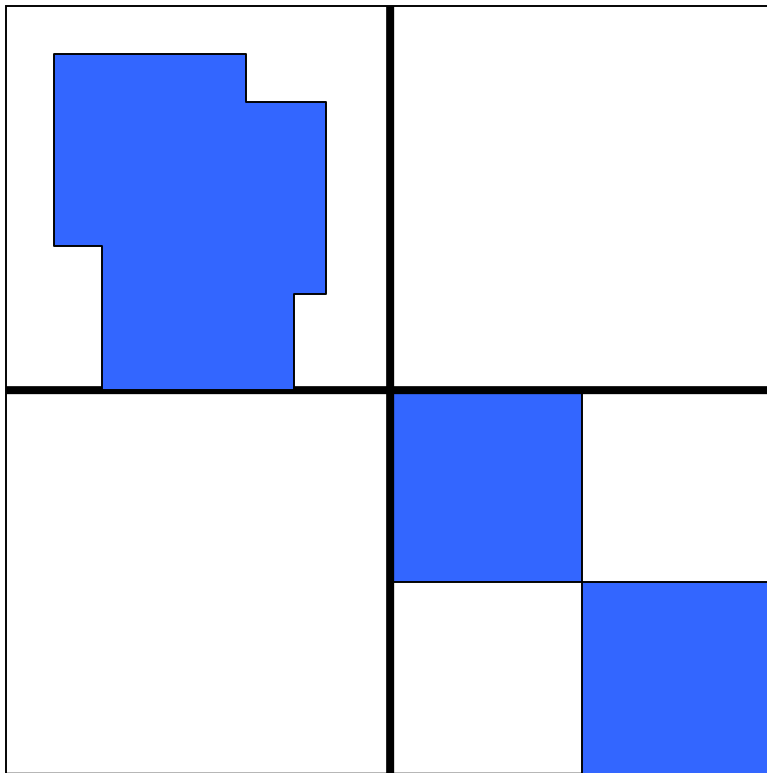


Equivalent quadtree

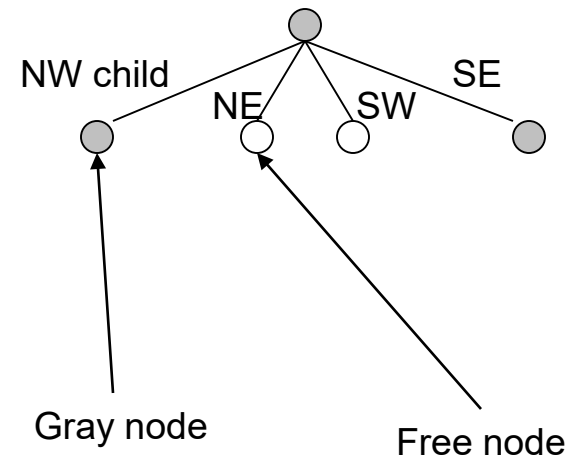


Quadtree Example

Space Representation

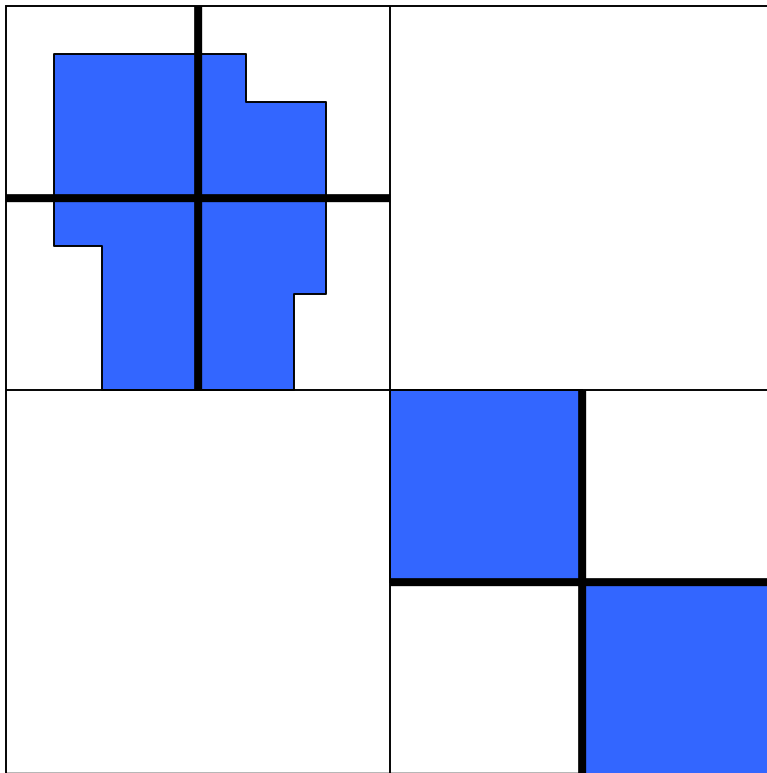


Equivalent quadtree

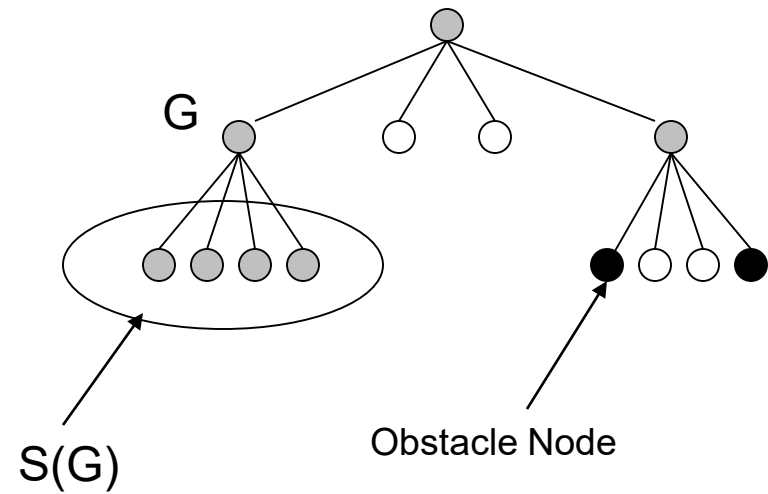


Quadtree Example

Space Representation

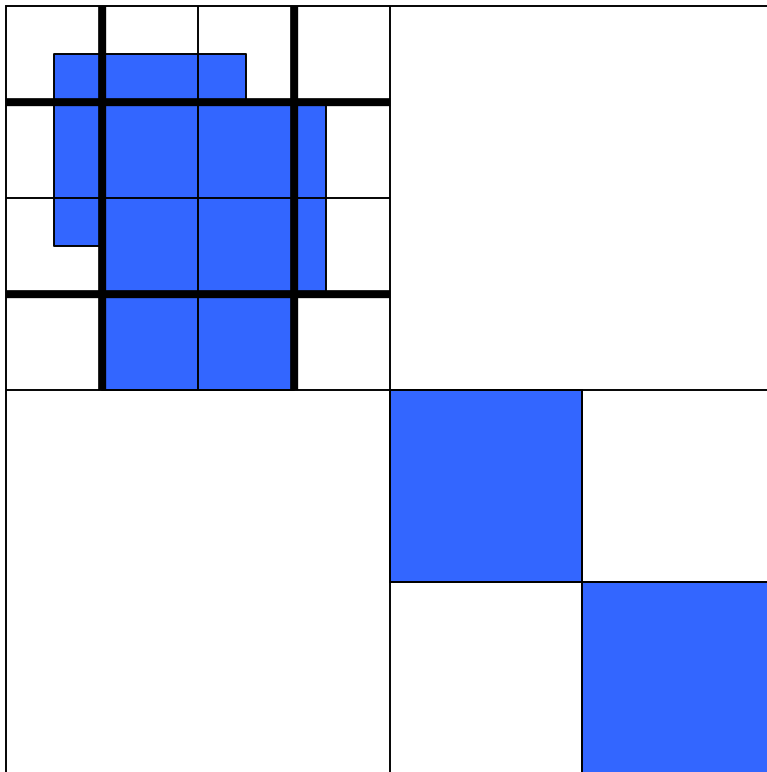


Equivalent quadtree

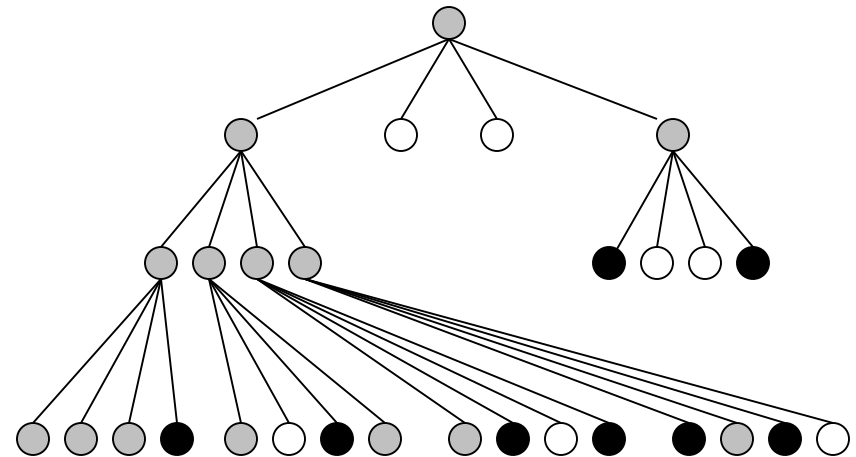


Quadtree Example

Space Representation



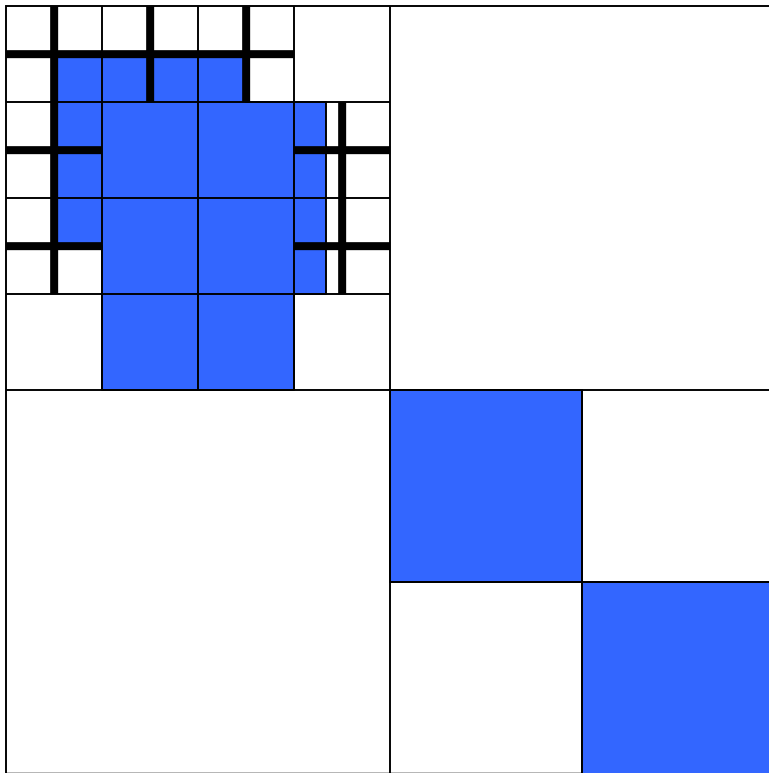
Equivalent quadtree



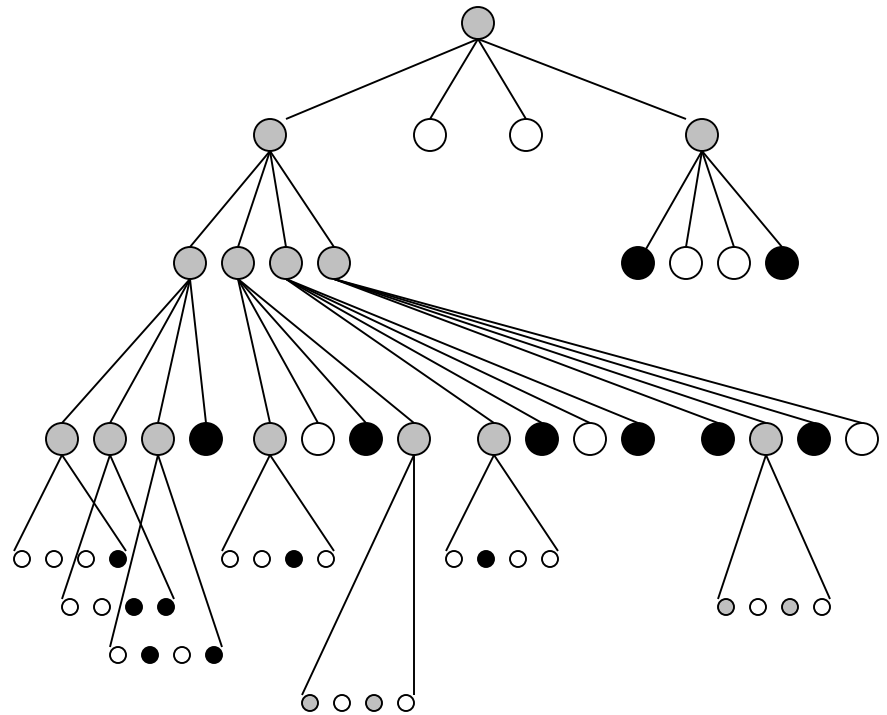
Each of these steps are examples of pruned quadtrees, or the space at different resolutions

Quadtree Example

Space Representation

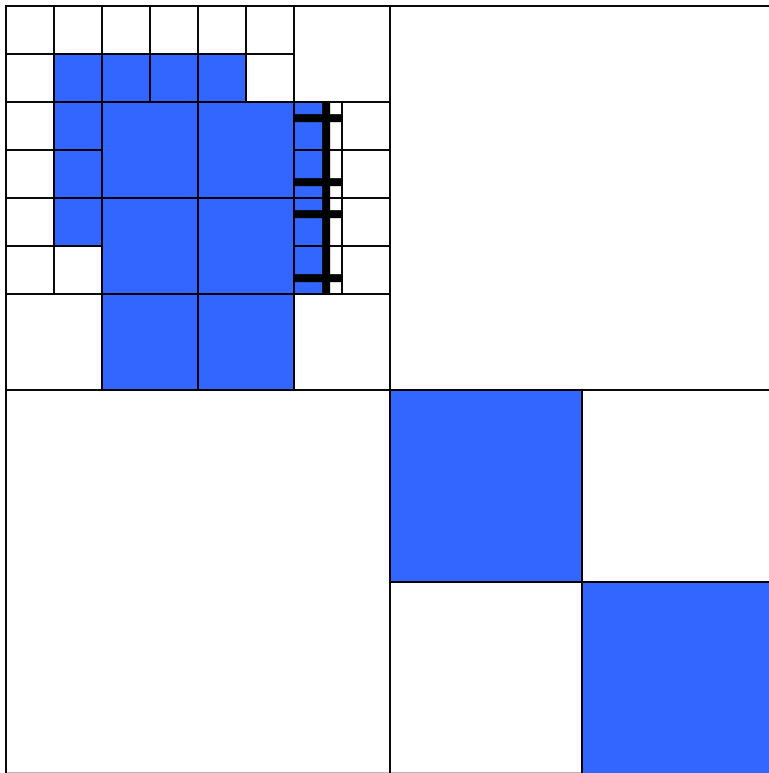


Equivalent quadtree

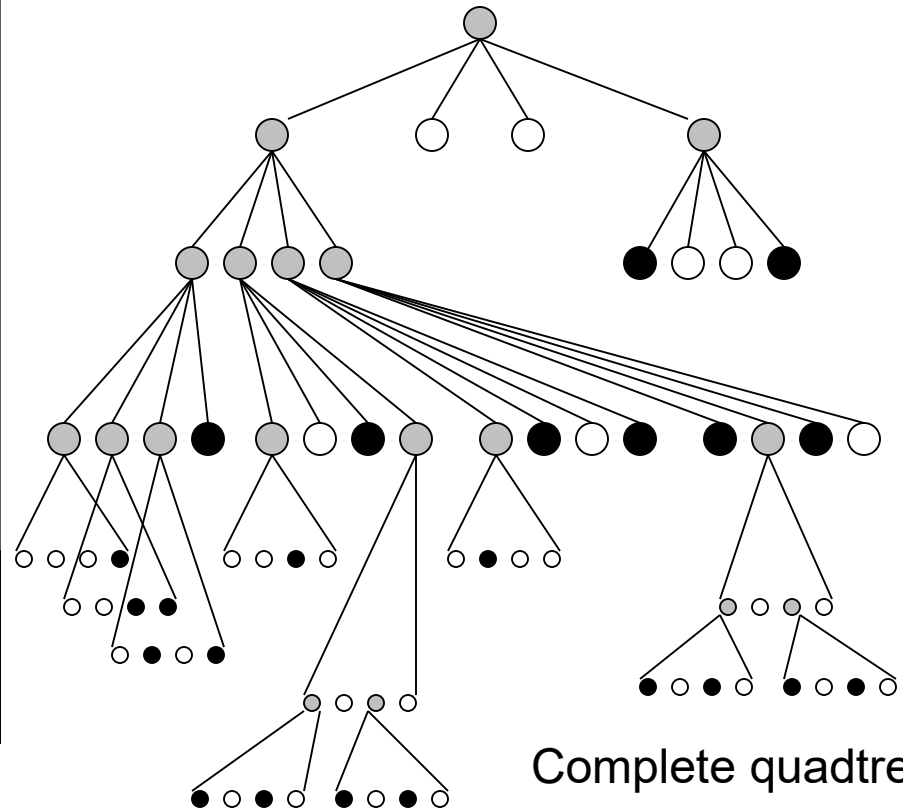


Quadtree Example

Space Representation



Equivalent quadtree



Complete quadtree

Planner Wish List

- Complete
- Handles narrow / tight passages.
- Bounded running time.
- Behavior well characterized.
- Low complexity
- Not too hard to understand
- Not too hard to implement
- Scalable

Motion Planning for Point Robots

■ Input

- Robot represented as a **point** in the **plane**
- Obstacles represented as polygons
- Initial and goal positions

■ Output

- A collision-free path between the initial and goal positions

continuous representation

(configuration space formulation)



discretization

(random sampling, processing critical geometric events)



graph searching

(breadth-first, best-first, A*)

Visibility Graph Method

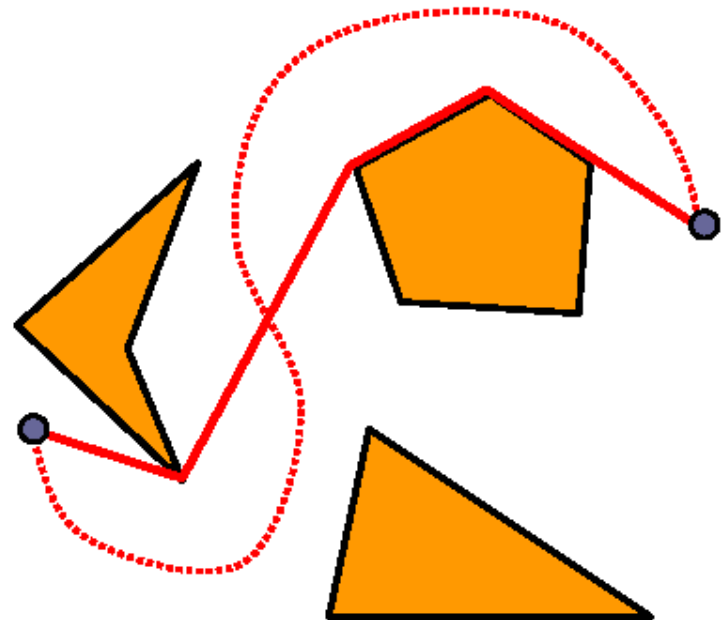
- **Observation:**

- If there is a collision-free path between two points, then there is a polygonal path that bends only at the obstacles vertices.

- **Why ?**

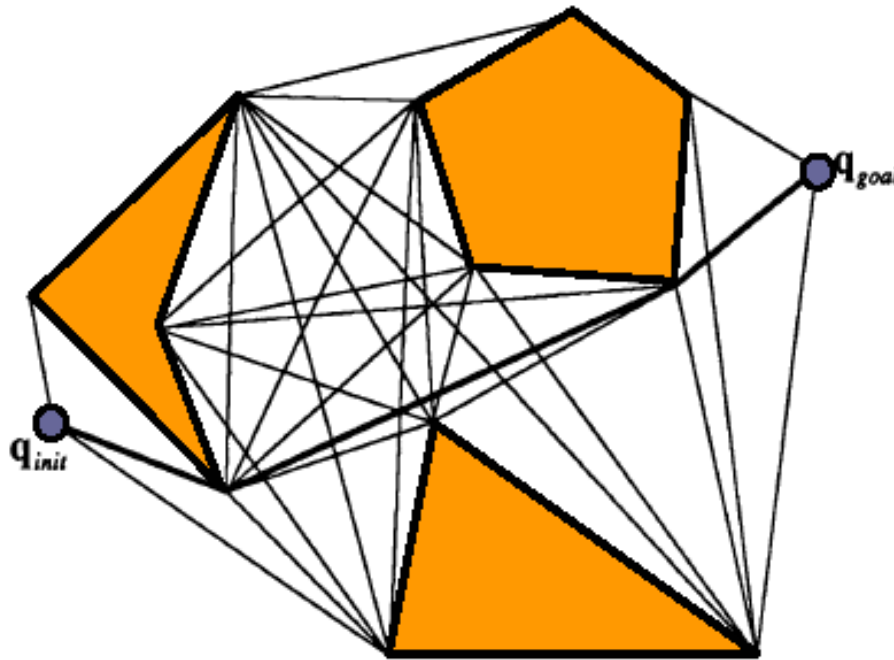
- Any collision-free path can be transformed into a polygonal path that bends only at the obstacle vertices.

- A **polygonal path** is a piecewise linear curve.



What is a Visibility Graph ?

- A **visibility graph** is a graph such that
 - Nodes: q_{init} , q_{goal} , or an obstacle vertex.
 - Edges: An edge exists between nodes u and v if the line segment between u and v is an obstacle edge or it does not intersect the obstacles.



Breadth First Search

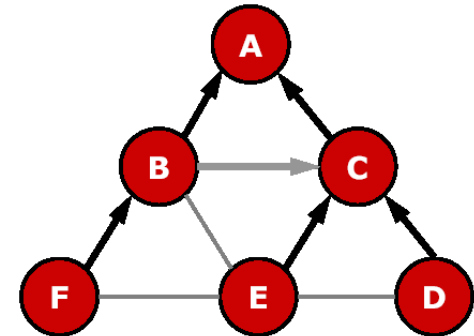
■ Input:

- q_{init} , q_{goal} , visibility graph G

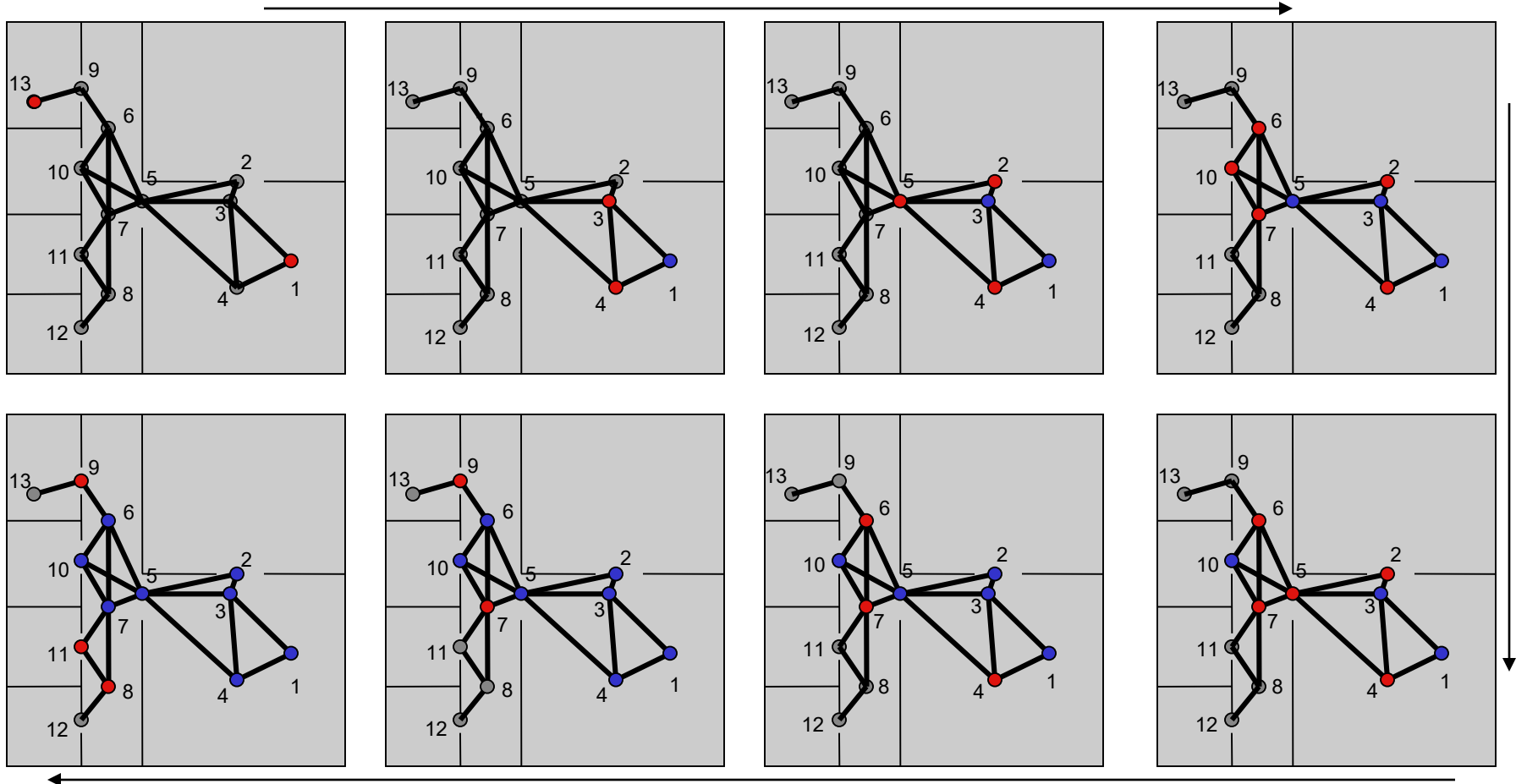
■ Output:

- A path between q_{init} and q_{goal}

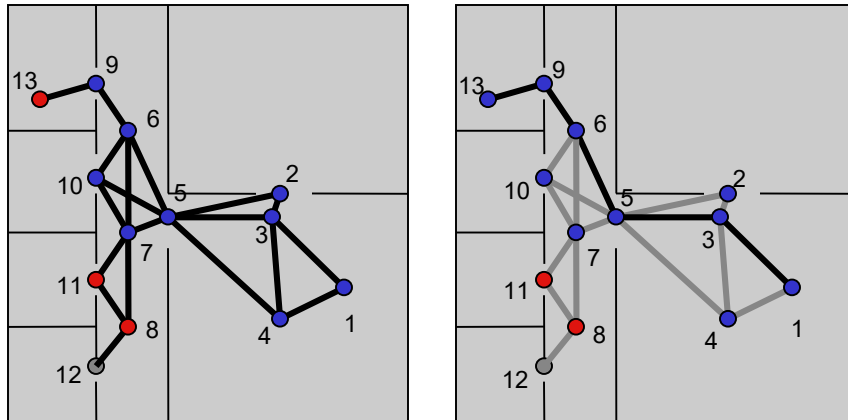
```
1: Q = new queue;
2: Q.enqueue( $q_{init}$ );
3: mark  $q_{init}$  as visited;
4: while Q is not empty
5:   curr = Q.dequeue();
6:   if curr ==  $q_{goal}$  then
7:     return curr;
8:   for each w adjacent to curr
10:    if w is not visited
11:      w.parent = curr;
12:      Q.enqueue(w)
13:    mark w as visited
```



A* Search (Straight-Line-Distance Heuristic)



A* Search (Straight-Line-Distance Heuristic)



Note that A* expands fewer nodes than breadth-first, but more than greedy

It's the price you pay for optimality

Keys are:

- Data structure for a node
- Priority queue for sorting open nodes
- Underlying graph structure for finding neighbours

Classic Motion Planning Approaches

■ Roadmap

- Represent the connectivity of the free space by a network of 1-D curves

■ Cell decomposition

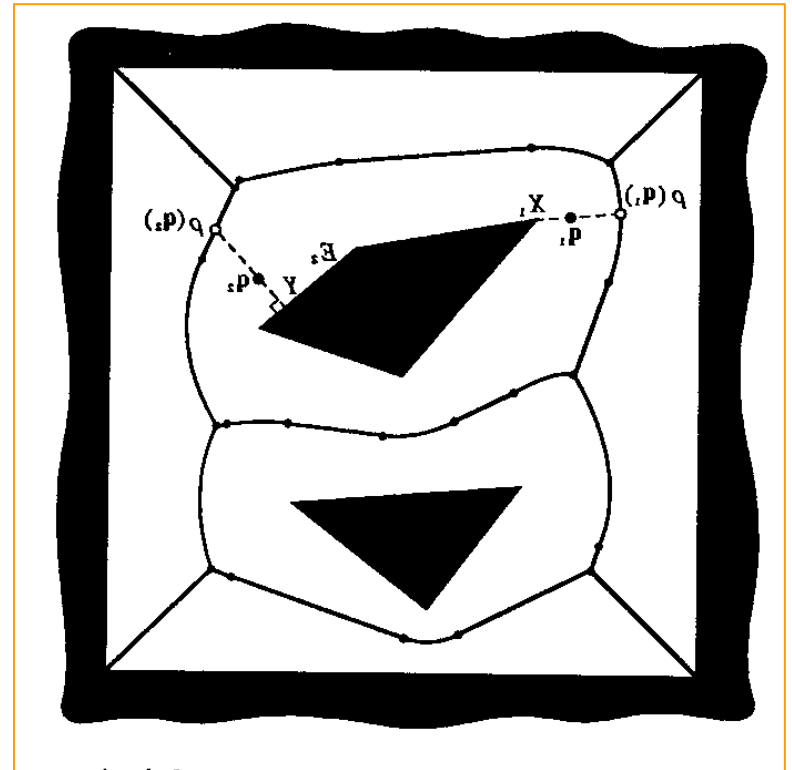
- Decompose the free space into simple cells and represent the connectivity of the free space by the adjacency graph of these cells

■ Potential field

- Define a potential function over the free space that has a global minimum at the goal and follow the steepest descent of the potential function

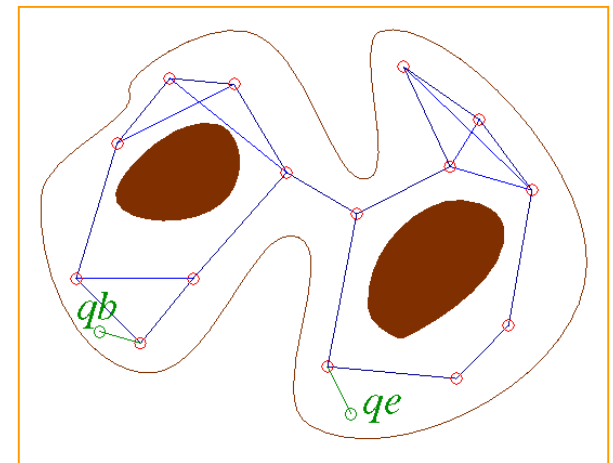
Motion Planning as a Computational Problem

- Goal: Characterize the connectivity of a space (e.g., the collision-free subset of configuration space)
- High computational complexity: Requires time exponential in number of degrees of freedom, or number of moving obstacles, or etc...
- Two main algorithmic approaches:
 - Planning by random sampling
 - **Planning by extracting criticalities**



Motion Planning as a Computational Problem

- Goal: Characterize the connectivity of a space (e.g., the collision-free subset of configuration space)
- High computational complexity: Requires time exponential in number of degrees of freedom, or number of moving obstacles, or etc...
- Two main algorithmic approaches:
 - **Planning by random sampling**
 - Planning by extracting criticalities



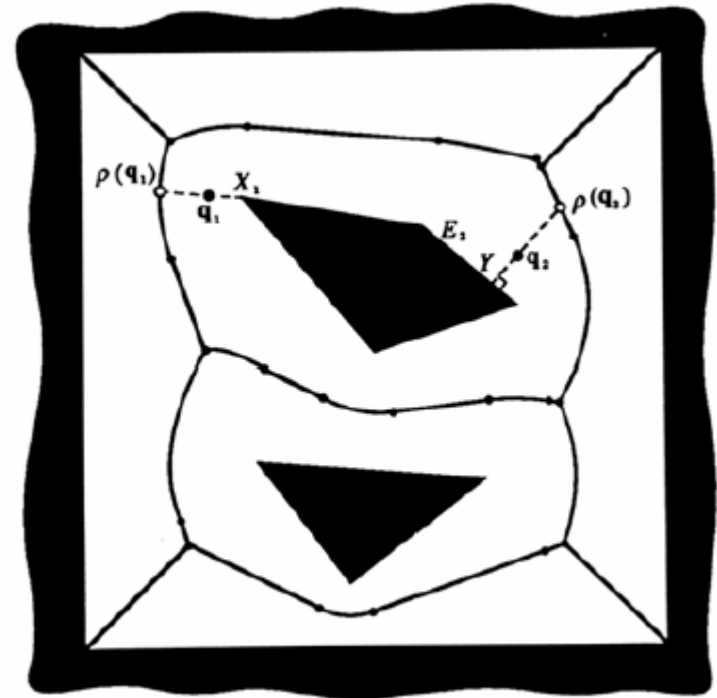
Roadmap

- Visibility Graph

- Shakey Project, SRI [Nilsson, 1969]

- Voronoi diagram

- Introduced by computational geometry researchers. Generate paths that maximizes clearance.
 - Applicable mostly to 2-D configuration spaces.
 - Space $O(n)$
 - Running time $O(n \log n)$



Classic Motion Planning Approaches

■ Roadmap

- Represent the connectivity of the free space by a
- network of 1-D curves

■ Cell decomposition

- Decompose the free space into simple cells and represent the connectivity of the free space by the adjacency graph of these cells

■ Potential field

- Define a potential function over the free space that has a global minimum at the goal and follow the steepest descent of the potential function

Cell Decomposition Methods

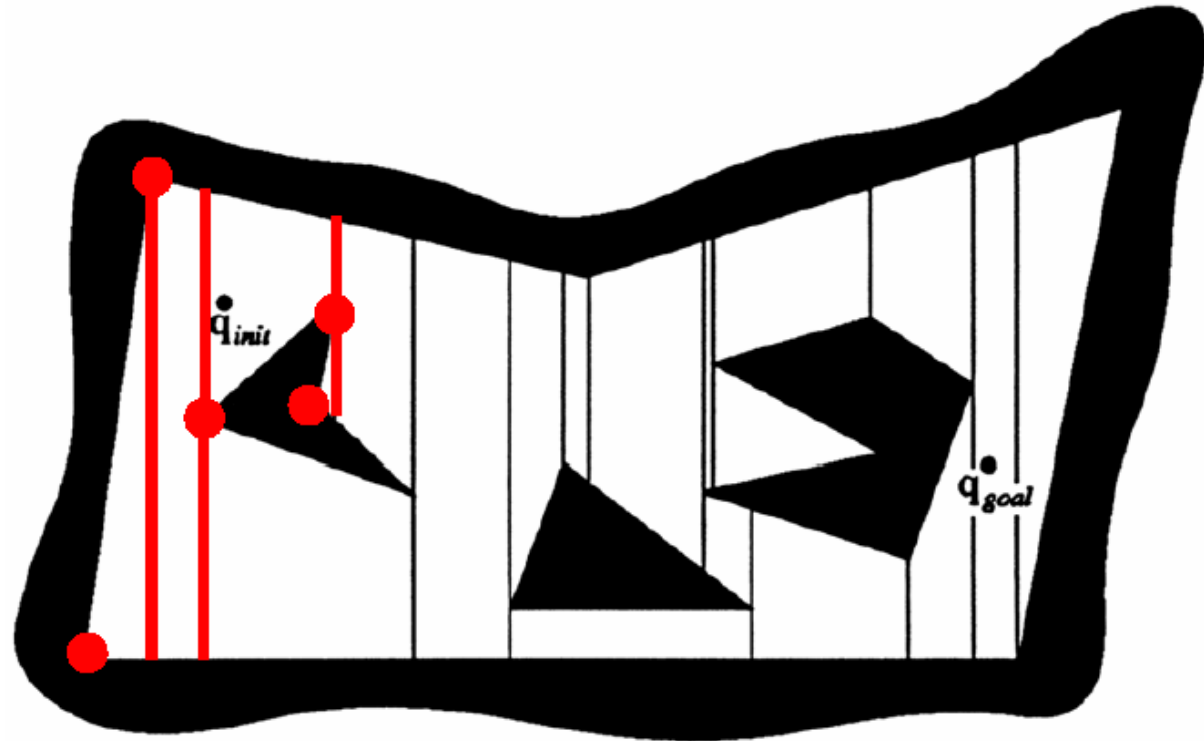
- **Exact cell decomposition**

- The free space F is represented by a collection of nonoverlapping simple cells whose union **is exactly** F

- Examples of cells: trapezoids, triangles

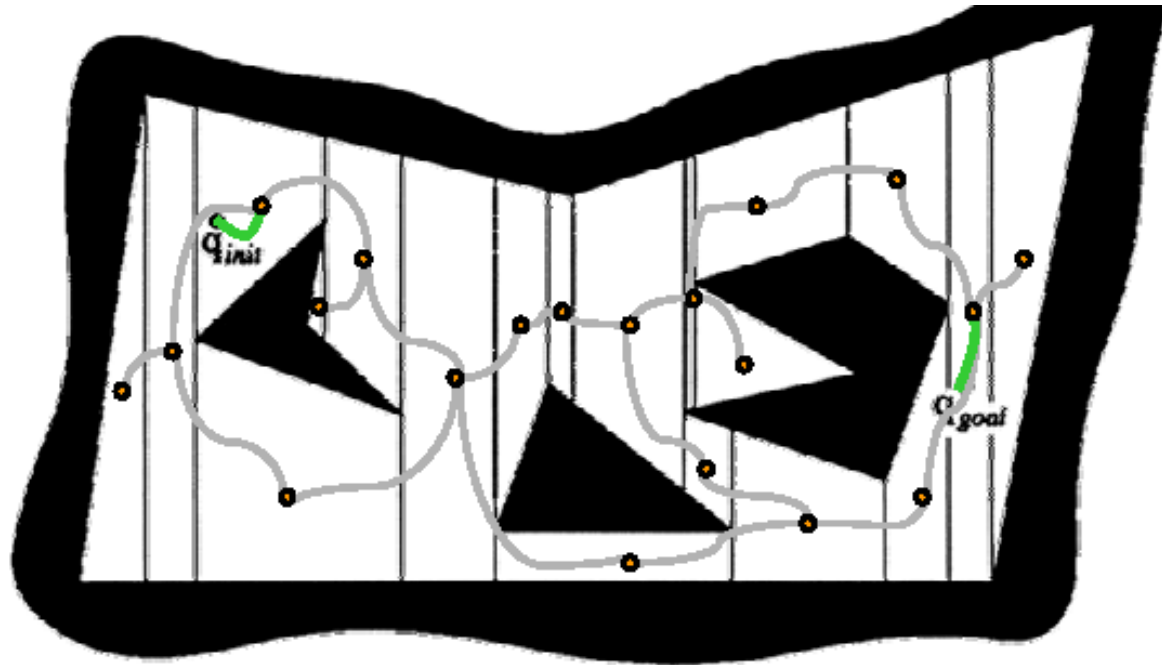
- Example: trapezoidal decomposition

- Sweep Line Algorithm



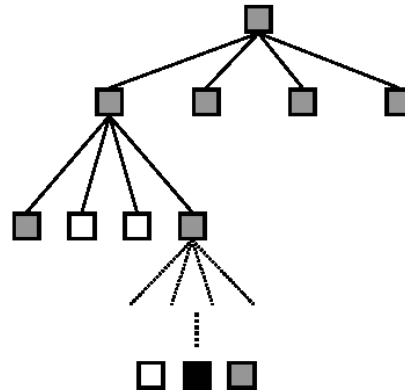
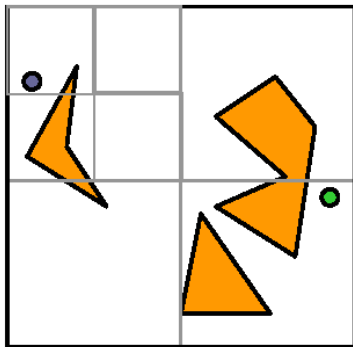
Computational Efficiency

- Running time $O(n \log n)$ by planar sweep
- Space $O(n)$
- Mostly for 2-D configuration spaces
- Adjacency graph
 - **Nodes:** cells
 - **Edges:** There is an edge between every pair of nodes whose corresponding cells are adjacent.



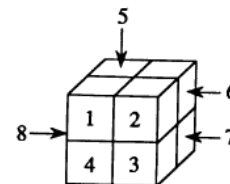
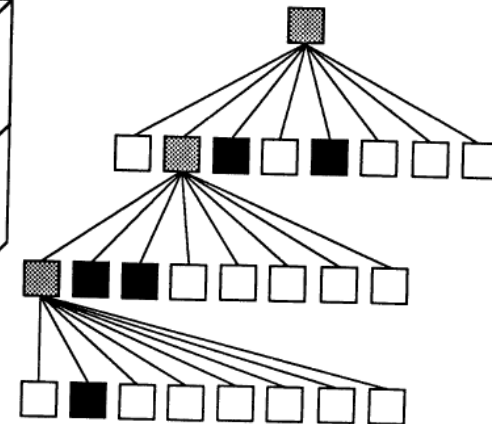
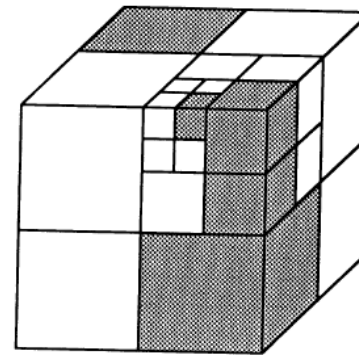
Cell Decomposition Methods

- Exact cell decomposition
- **Approximate cell decomposition**
 - Freespace F is represented by a collection of non-overlapping cells whose union is **contained** in F .
 - Cells usually have simple, regular shapes, e.g., rectangles, squares.
 - Facilitate hierarchical space decomposition
 - 2D Quadtree



□ empty ■ mixed ■ full

3D Octree



□ EMPTY cell ■ MIXED cell ■ FULL cell

Classic Motion Planning Approaches

■ Roadmap

- Represent the connectivity of the free space by a network of 1-D curves

■ Cell decomposition

- Decompose the free space into simple cells and represent the connectivity of the free space by the adjacency graph of these cells

■ Potential field

- Define a potential function over the free space that has a global minimum at the goal and follow the steepest descent of the potential function

Potential Fields

- Initially proposed for real-time collision avoidance [Khatib, 1986]. Hundreds of papers published on this topic.
- A potential field is a scalar function over the free space.
- To navigate, the robot applies a force proportional to the negated gradient of the potential field.
- A **navigation function** is an ideal potential field that
 - has global minimum at the goal
 - has no local minima (hard to obtain)
 - grows to infinity near obstacles
 - is smooth

Attractive and repulsive fields

$$F_{\text{att}} = -k_{\text{att}} (x - x_{\text{goal}})$$

$$F_{\text{rep}} = \begin{cases} k_{\text{rep}} \left(\frac{1}{\rho} - \frac{1}{\rho_0} \right) \frac{1}{\rho^2} \frac{\partial \rho}{\partial x} & \text{if } \rho \leq \rho_0, \\ 0 & \text{if } \rho > \rho_0 \end{cases}$$

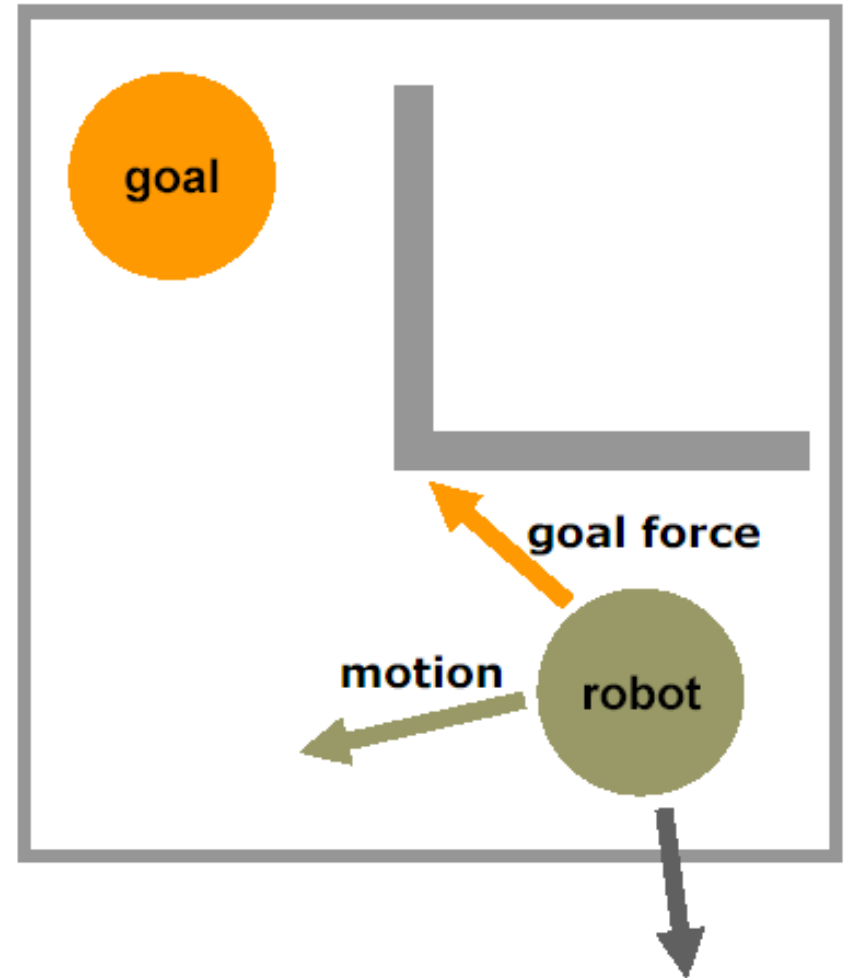
$k_{\text{att}}, k_{\text{rep}}$: positive scaling factors

x : position of the robot

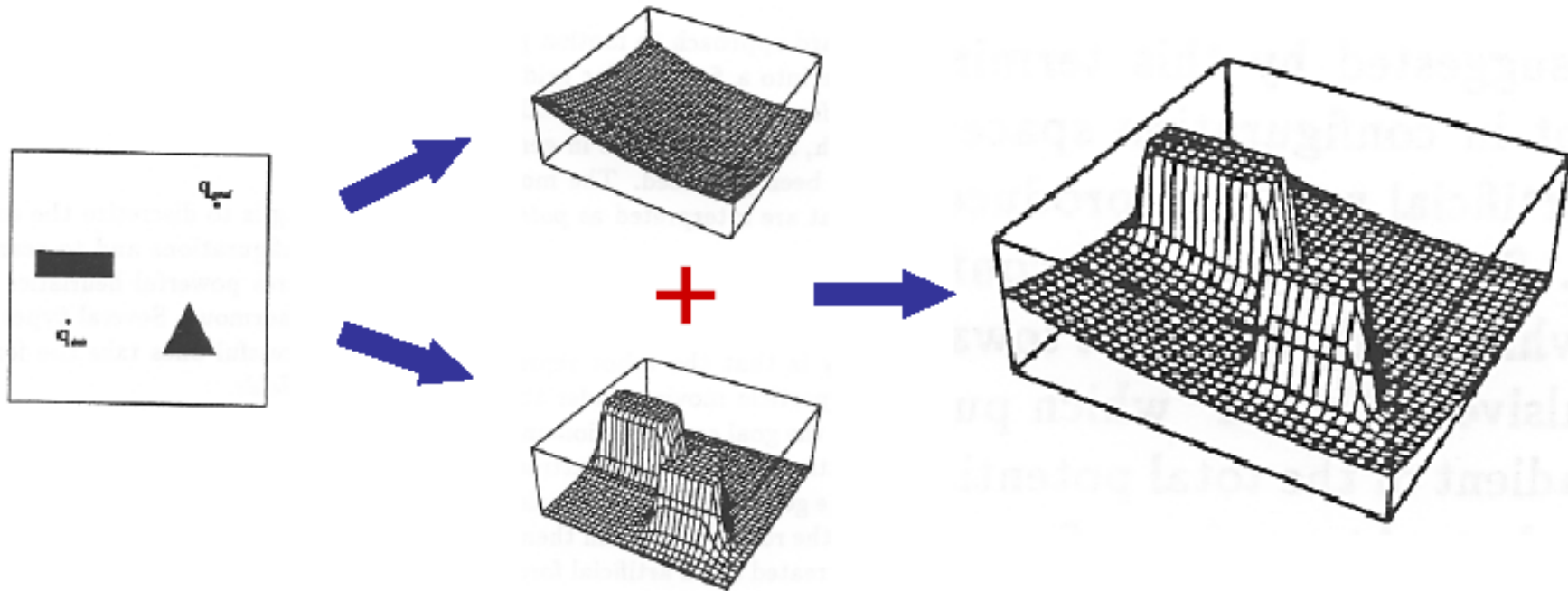
ρ : distance to the obstacle

ρ_0 : distance of influence

[Khatib, 1986]

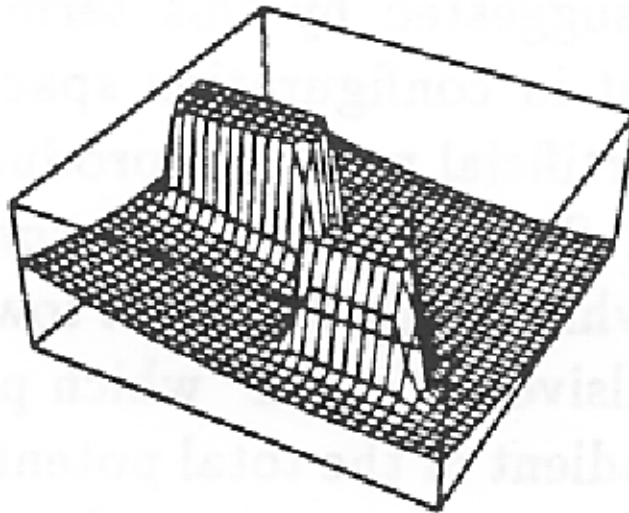


Algorithm in pictures

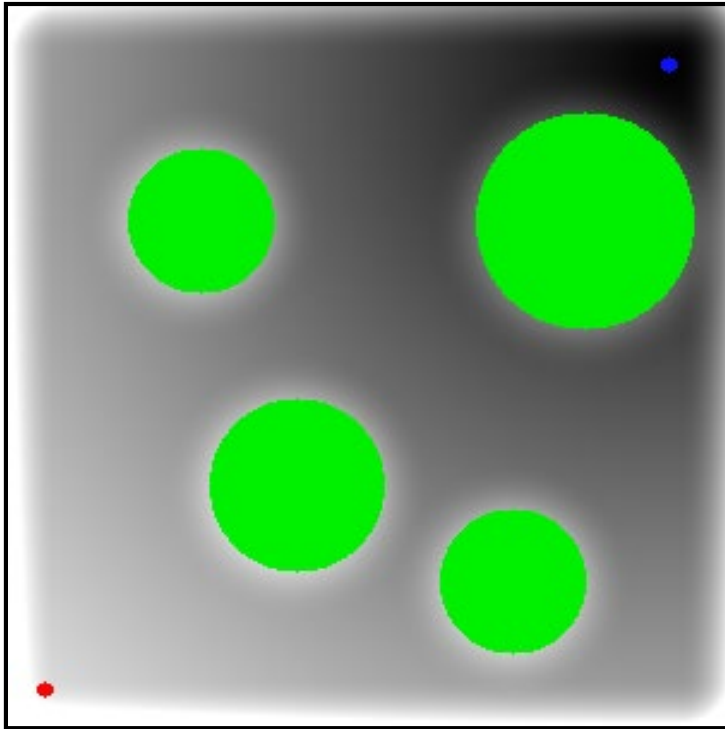


Motion Planning for Point Robots

- Place a regular grid G over the configuration space
- Compute the potential field over G
- Search G using a best-first algorithm with potential field as the heuristic function
- Local minima: What can we do?
 - Escape from local minima by taking random walks
 - Build an ideal potential field – navigation function – that does not have local minima
 - Can such an ideal potential field be constructed efficiently in general?



Potential Field

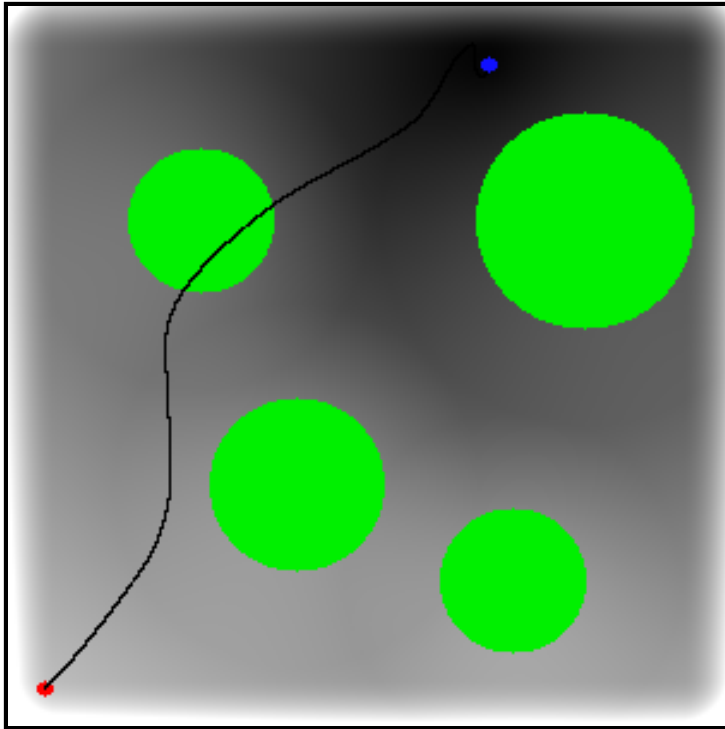


- Red is start point, blue is goal
- This used a quadratic field strength around the obstacles
- Note that the boundaries of the world also contribute to the field

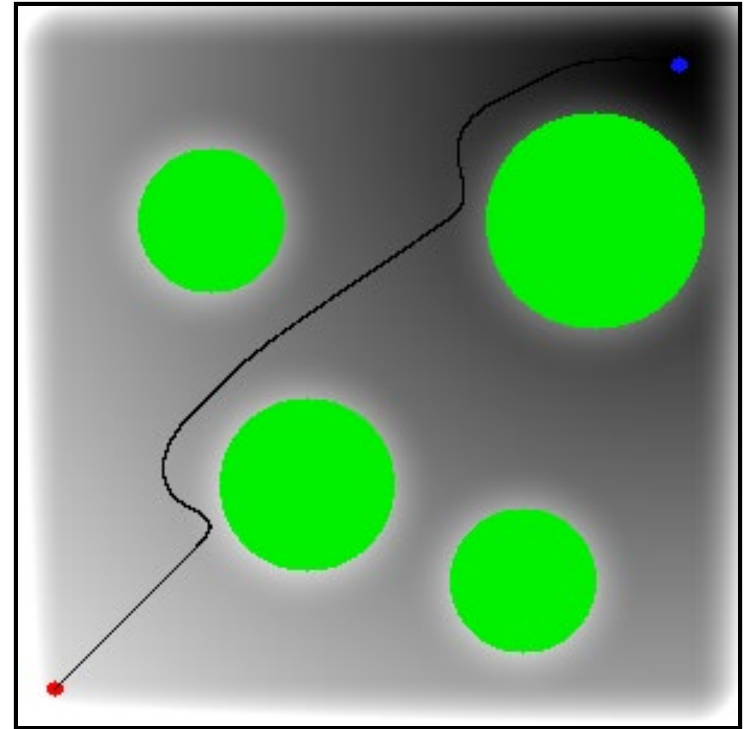
Potential Field Problems

- There are many parameters to tune
 - Strength of the field around each obstacle
 - Function for field strength around obstacle
 - Steepness of force toward the goal
- Goals conflict
 - High field strength avoids collisions, but produces big forces and hence unnatural motion
- Local minima cause huge problems

Problems

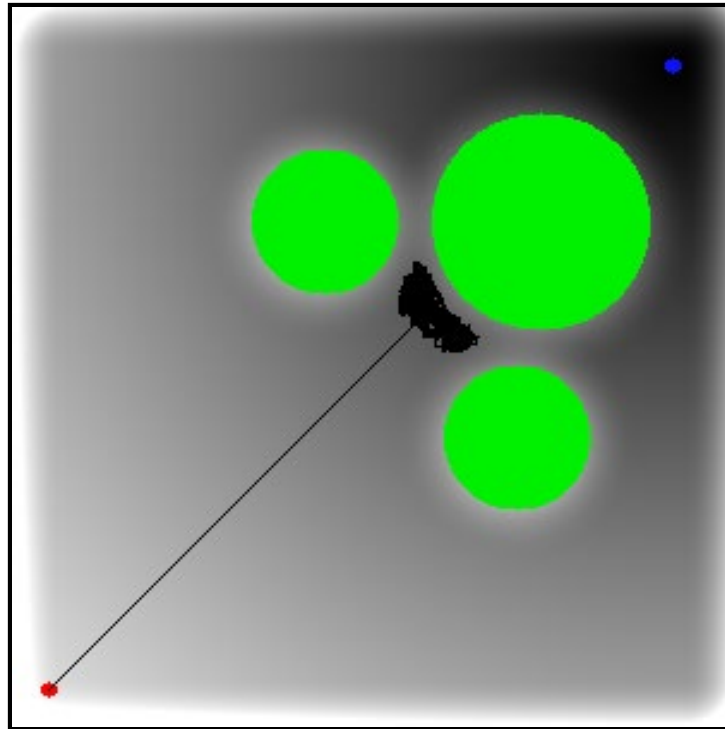


Field too weak



Field too strong

Local Minima Example



The Local Minima Problem

- Recall, path planning can be viewed as optimization
- Potential field planning is gradient descent optimization
- The biggest problem with gradient descent is that it gets stuck in local minima
- Potential field planning suffers from exactly the same problem
- Must have a way to work around this
 - Go back if a minima is found, and try another path
- With what sorts of environments will potential fields work best?

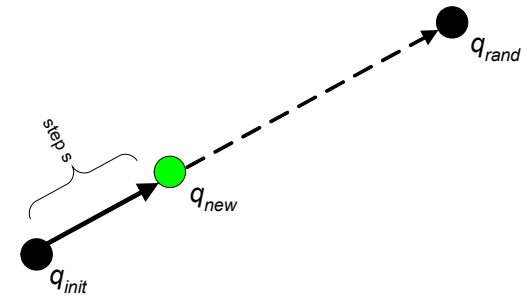
Rapidly Exploring Random Tree (RRT) Algorithm

- **Init:** Add q_{init} to the RRT
- 1. Generate a random position q_{rand} in C_{free}
- 2. Find the node q_{near} in the so far built-ob RRT which is the nearest to the random position q_{rand}
- 3. Check if you can reach a node q_{new} starting from q_{near} in direction to q_{rand} in a fixed distance s without collision.
If q_{new} can be reached without collision add the new node with an edge from q_{near} to q_{new} to the RRT.
- 4. Check if q_{new} is near enough to q_{goal} to reach it.
If the distance is too big, continue with step 1.
- 5. If the distance is below a certain value try to connect q_{new} with q_{goal} . If there is a collision on the way from q_{new} to q_{goal} , continue with step 1.
- 6. If q_{goal} could be reached without collision add it to the RRT and connect it with q_{new} .

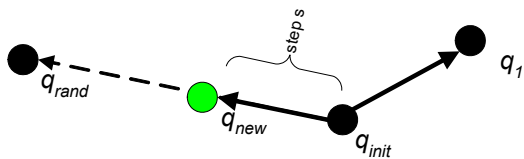
Now the path from q_{init} to q_{goal} can be determined by going backwards starting from q_{goal} .

RRT Algorithm

C-Space

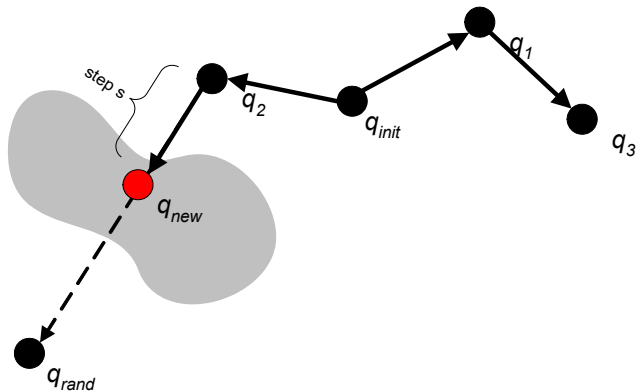


C-Space

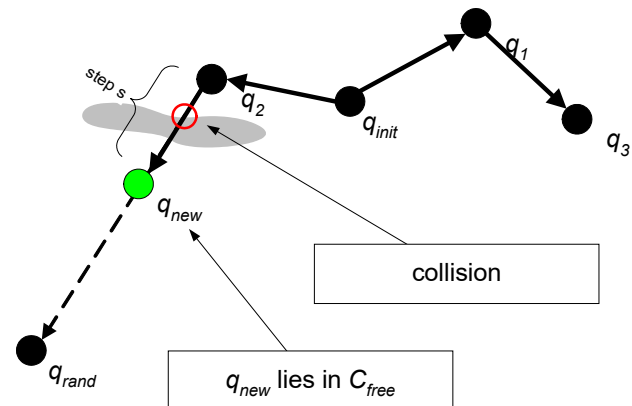


RRT Algorithm

C-Space

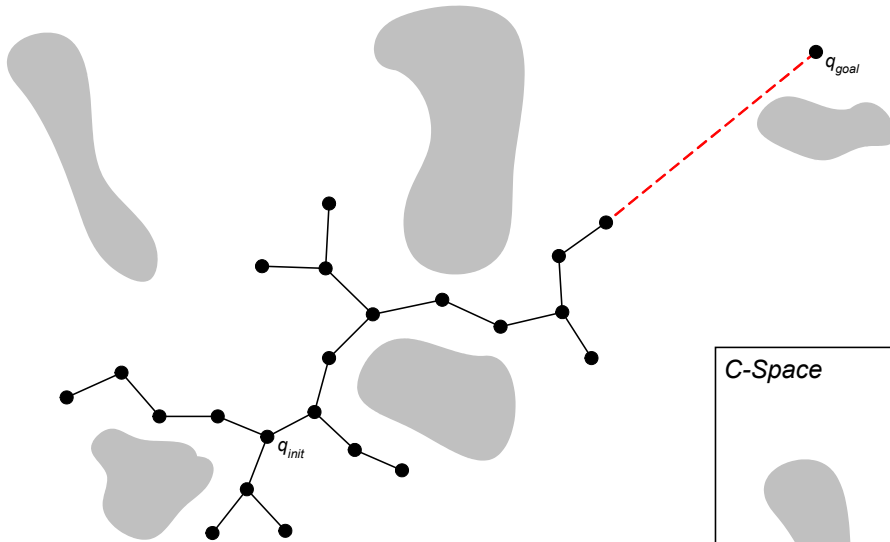


C-Space

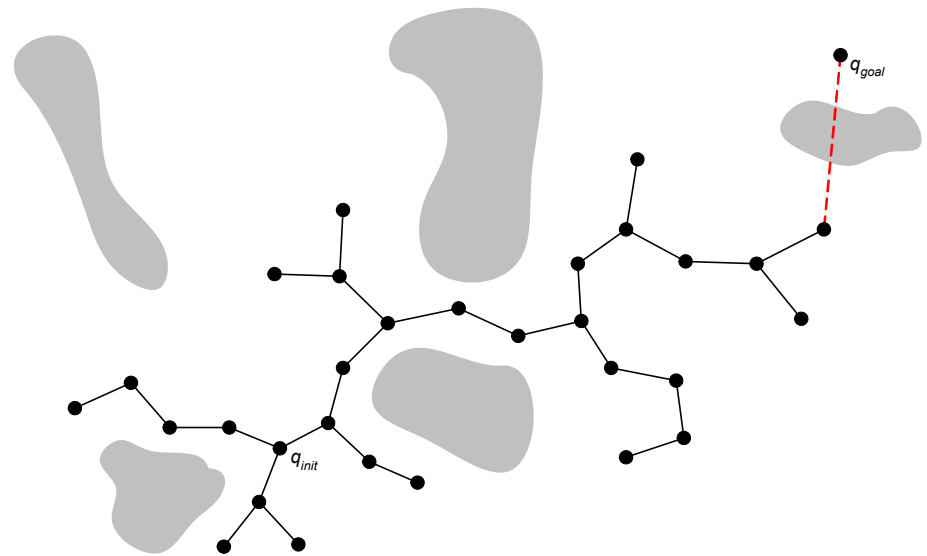


RRT Algorithm

C-Space

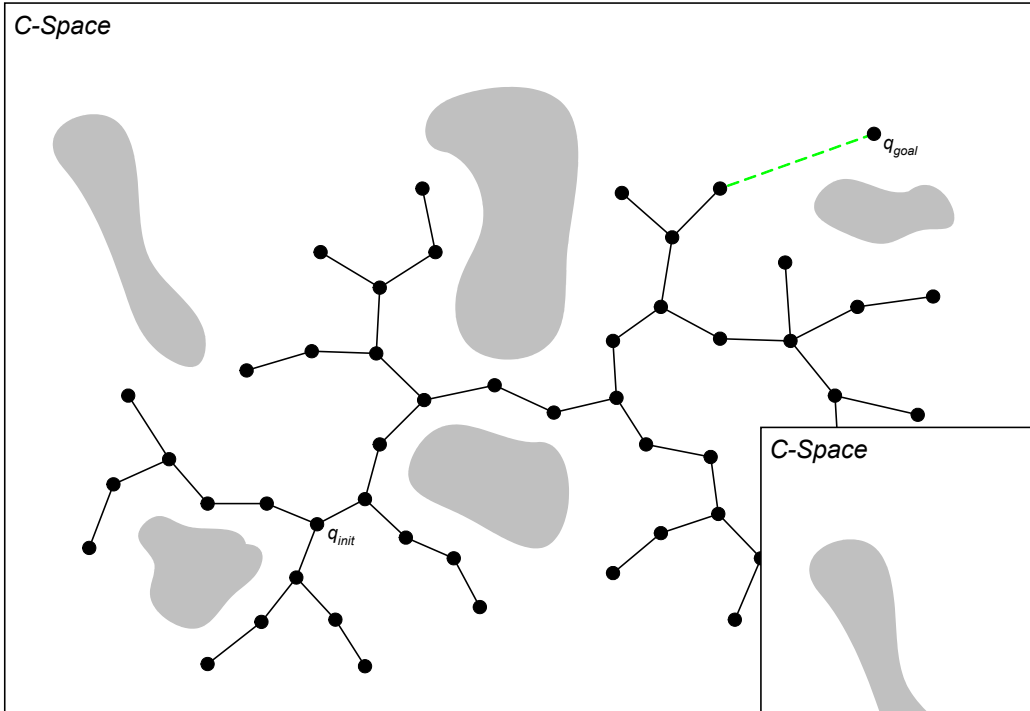


C-Space

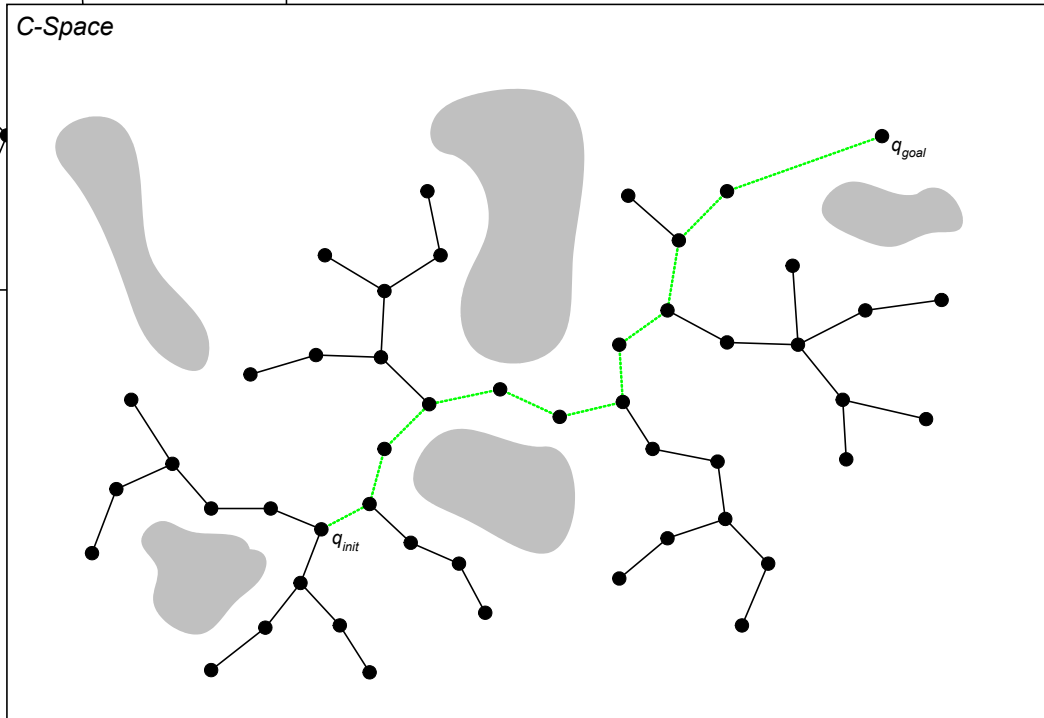


RRT Algorithm

C-Space

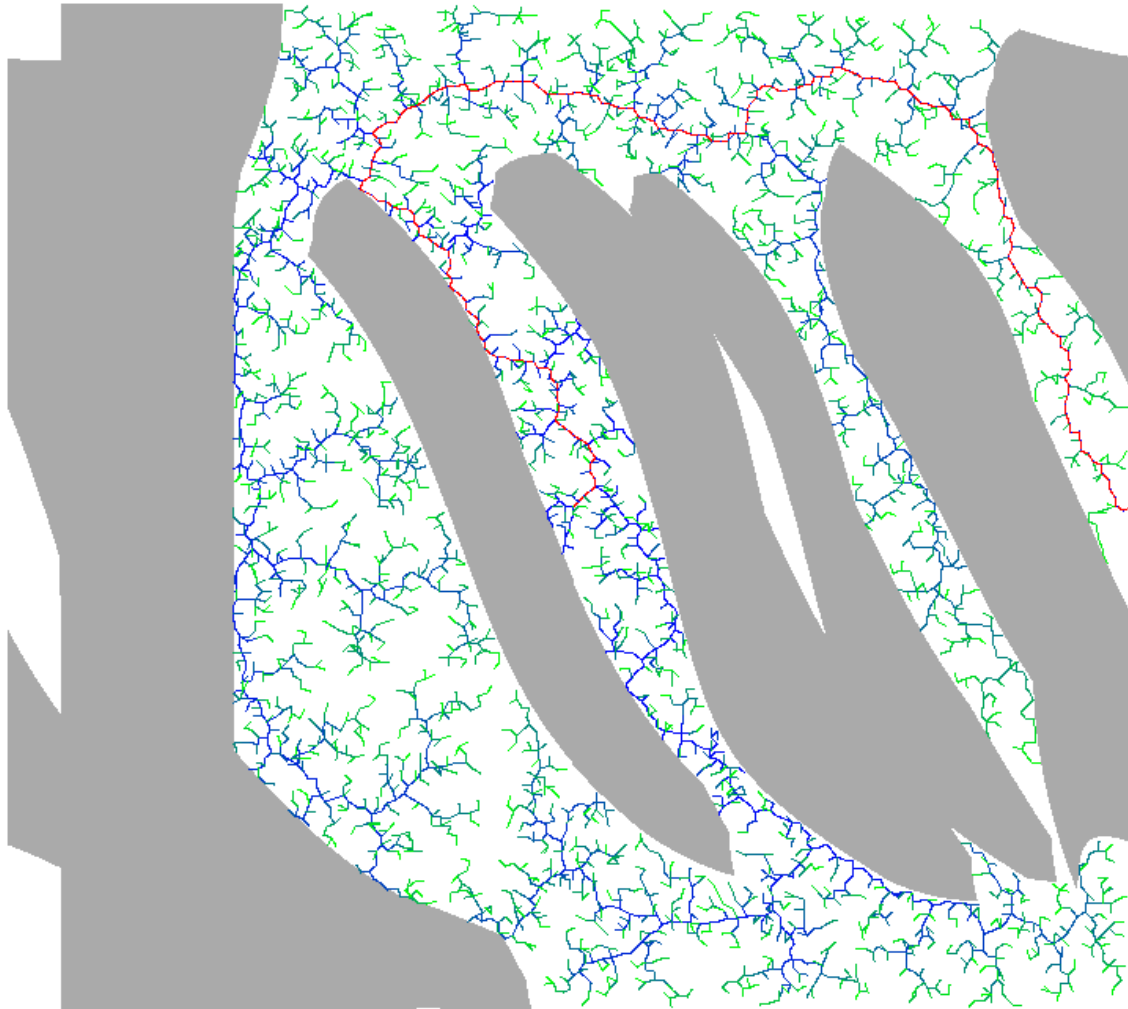


C-Space

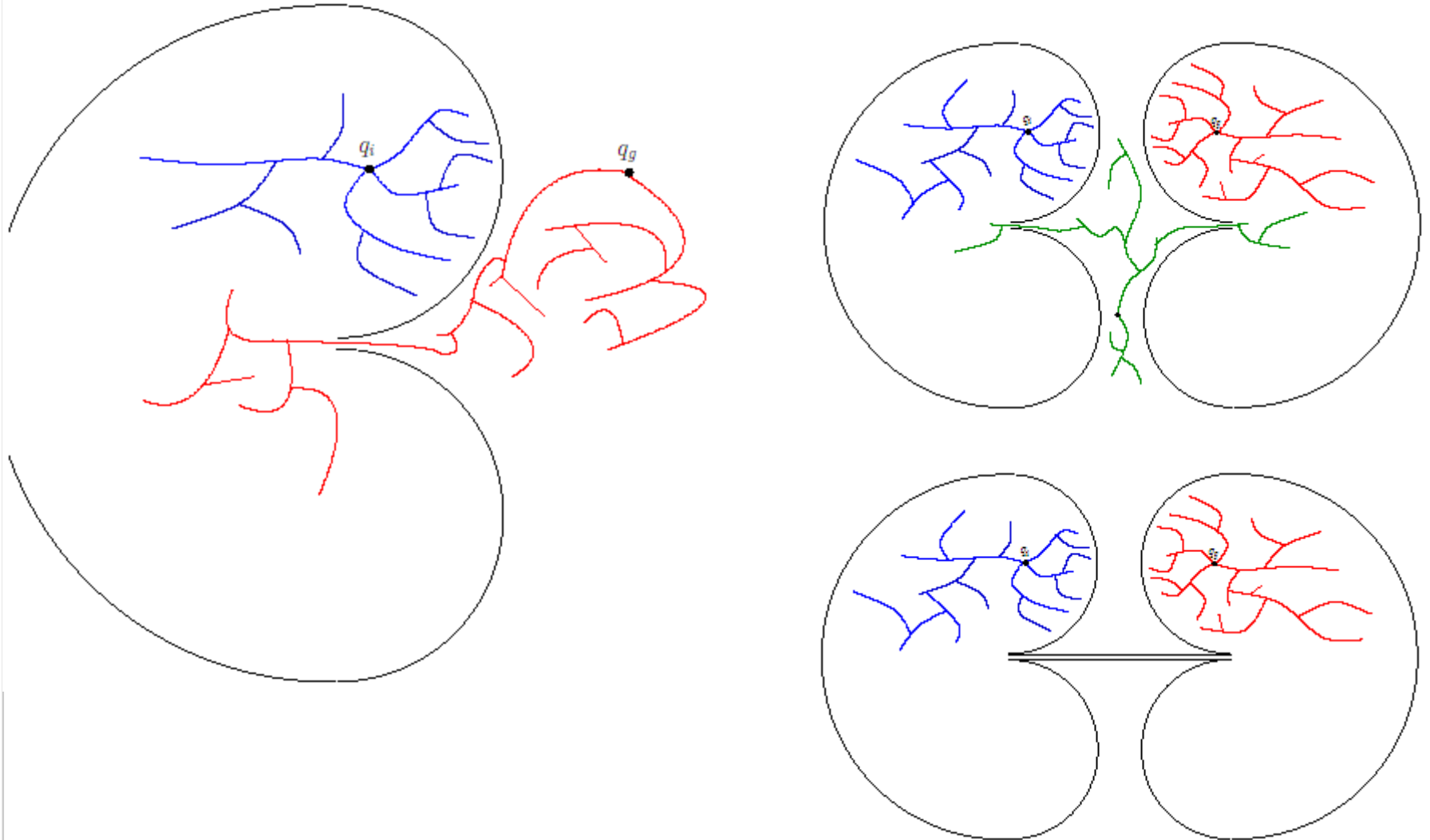


RRT Algorithm

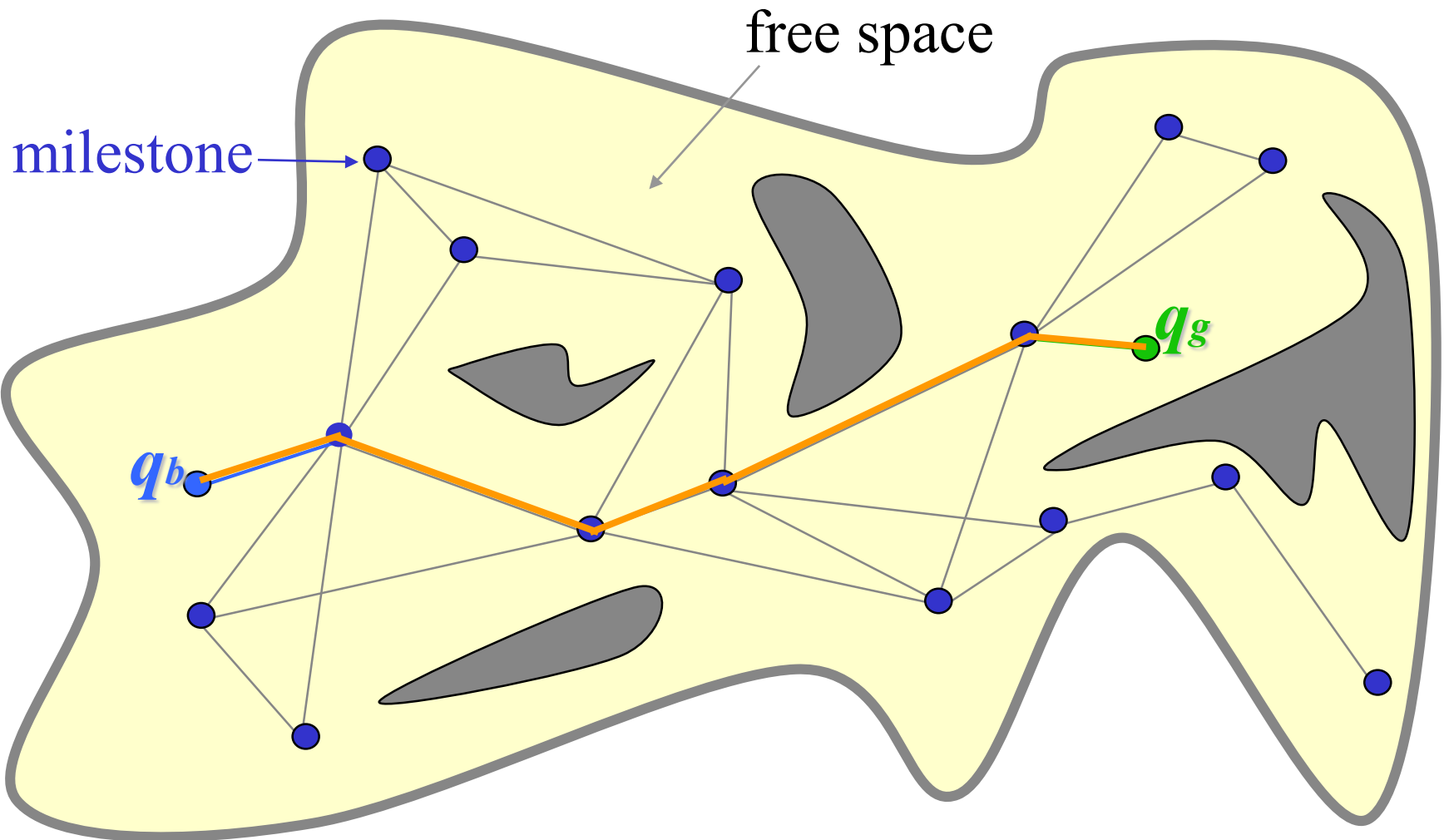
colors represent the growth of the RRT over time



Bug traps motivates bidirectional search



Principle of Randomized Planning



in theory

- is **probabilistically complete**, i.e., whenever a solution exists, the probability that it finds one tends toward 1 as the number N of milestones increases
- under rather general hypotheses, the rate of convergence is exponential in the number N of milestones, i.e.:

$$\text{Prob}[\text{failure}] \sim \exp(-N)$$

in practise

- are fast
- deal effectively with many-dof robots
- are easy to implement
- have solved complex problems

Roadmap Construction

We shall construct the roadmap $R = (V, E)$ using the following algorithm:

1. Start with an empty graph ($V \leftarrow \emptyset, E \leftarrow \emptyset$).
2. Perform the following loop:
 - Select a random free configuration $c \in C_{\text{free}}$.
 - Randomly pick a set of neighboring configurations $V_c \subseteq V$.
 - $V \leftarrow V \cup \{c\}$
 - For each $q \in V_c$, in increasing order of $D(c, q)$, check if there is a free local path between c and q .
If there exists such a path, add the edge: $E \leftarrow E \cup \{(c, q)\}$

Some Open Questions

How to create random free configurations?

It is possible to uniformly pick up a configuration and check if it is free.

How many neighbors should be chosen for each new configuration (the size of V_c)?

Pick up the closest neighbours.

How to define the distance function (in order to compute $D(c, q)$)?

This depends on the nature of the problem ...

How to check if there exists a local path between two neighboring configurations?

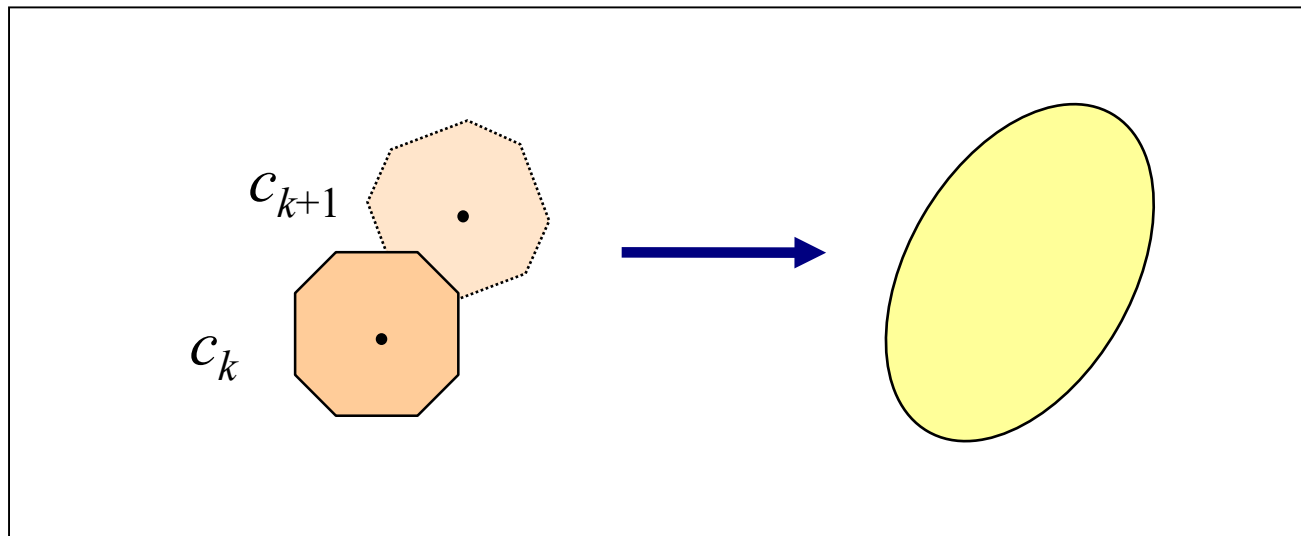
We must rely on a local planner (preferably a deterministic one).

The Local Planner

Given two free configurations c' and c'' , subdivide the straight line between them into m segments at the intermediate points:

$$c_0 = c', c_1, \dots, c_m = c''.$$

Try to connect each pair of adjacent point c_k and c_{k+1} . This is done by inflating the robot, and the path existence query is thus reduced to an interference query.



Roadmap Expansion

After constructing the initial roadmap $\mathcal{R} = (V, E)$ we can use the following algorithm to improve its connectivity:

1. Pick up a set of “problematic” configurations $U \subseteq V$.
2. For each configuration $u \in U$.
 - Compute a random-walk track from u until reaching a new configuration c .
 - $V \leftarrow V \cup \{c\}$
 - Connect: $E \leftarrow E \cup \{(u, c)\}$ and store the path with this edge (as this path was created randomly).
 - Try to connect c with neighboring configurations that belong to different connected components of the roadmap.
3. Remove small connected components from the roadmap.

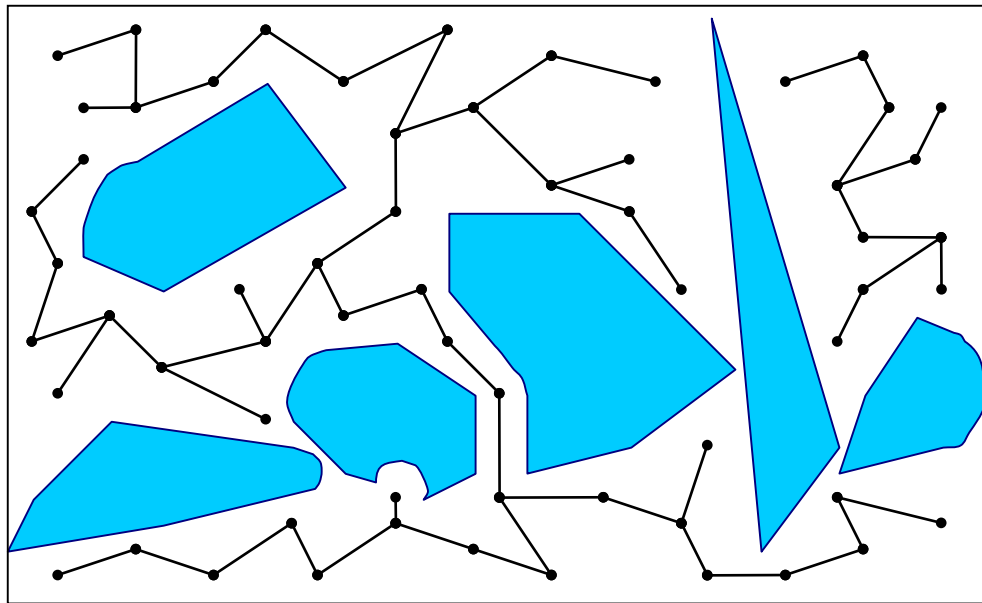
Answering Queries

Given a roadmap $\mathcal{R} = (V, E)$ with a start and target free configurations s and t :

1. Locate a vertex $s' \in V$ near s and try to connect s and s' .
Report on failure if it is impossible to locate such s' .
2. Locate a vertex $t' \in V$ near t and try to connect t and t' .
Report on failure if it is impossible to locate such t' .
3. In case of success, find a path between s' and t' in the roadmap. **If the two vertices are not connected, report on failure.**
4. If all went well, return the path $s \rightsquigarrow s' \rightsquigarrow t' \rightsquigarrow t$.

Analysis of Roadmaps

To have a better understanding of the capabilities of the probabilistic roadmaps, we should be able to estimate the number of configurations we should sample (or roadmap vertices) in order to capture the connectivity of the configuration space.



Path Planning in the Real World

Real World Robots

- Have inertia
- Have limited controllability
- Have limited sensors
- Face a dynamic environment
- Face an unreliable environment

Static planners like PRM are not sufficient

Extensions to the Basic Problem

- Moving obstacles
- Multiple robots
- Movable objects
- Deformable objects
- Goal is to gather data by sensing
- Nonholonomic constraints
- Dynamic constraints
- Optimal planning
- Uncertainty in control and sensing

Two Approaches to Path Planning

Kinematic: only concerns the motion, without regard to the forces that cause it

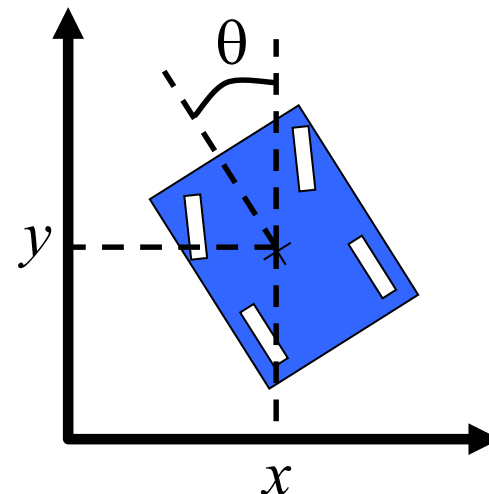
- Performs well for systems where position can be controlled directly
- Does not work well for systems with significant inertia

Kinodynamic: incorporates dynamic constraints

- Plans velocity as well as position

Configuration space represents the position and orientation of a robot
Sufficient for static planners like PRM

Example: Steerable car
Configuration space
 (x, y, q)



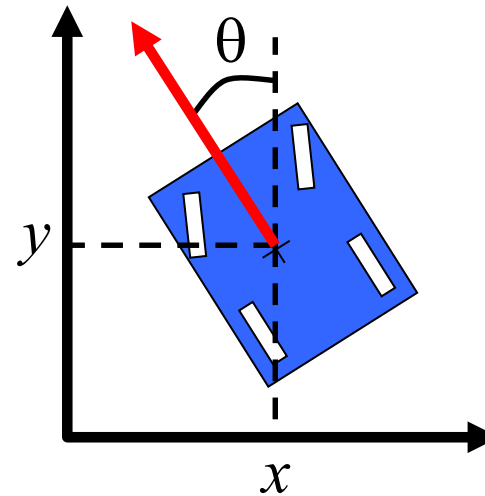
Representing Dynamic State

- State space incorporates the dynamic state of a robot

Example: Steerable car

State space

$$X = (x, y, \theta, \dot{x}, \dot{y}, \dot{\theta})$$



Working in state space allows planner to incorporate dynamic constraints on path

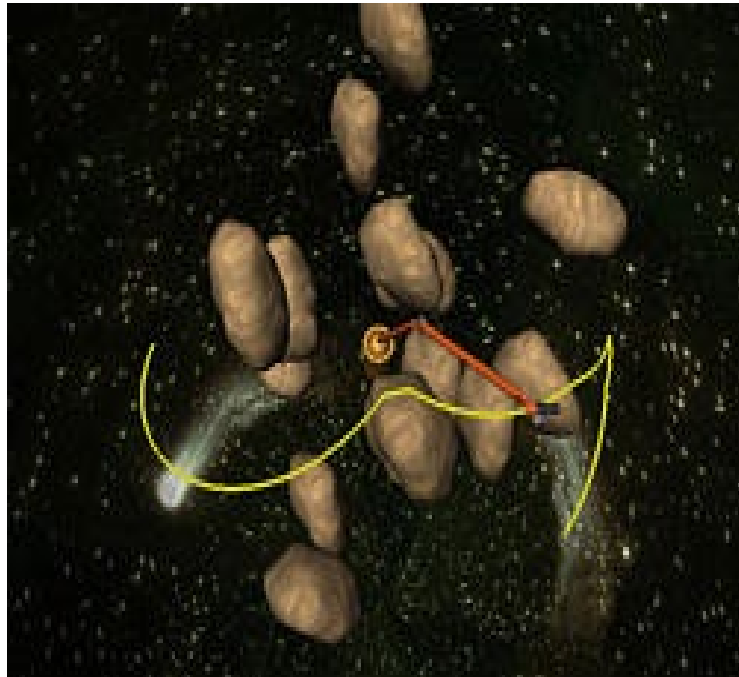
- *Examples:* maximum velocity, minimum turning radius

Working in state space doubles the dimensionality of the planning problem

- Math becomes exponentially harder

Planning Amidst Moving Obstacles

- Moving Obstacles Planner (MOP):
- A PRM extension that accounts for both kinematic and dynamic constraints of a robot navigating amidst moving obstacles
- We discuss: Kinodynamic motion planning amidst moving obstacles with known trajectories
- Example: Asteroid avoidance problem



Asteroid Avoidance Problem

Autonomous Vehicle
"Spacecraft"



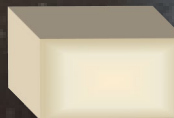
Moving Obstacles
"Asteroids"



Known trajectories



❖ Path-planning among moving obstacles with known trajectories



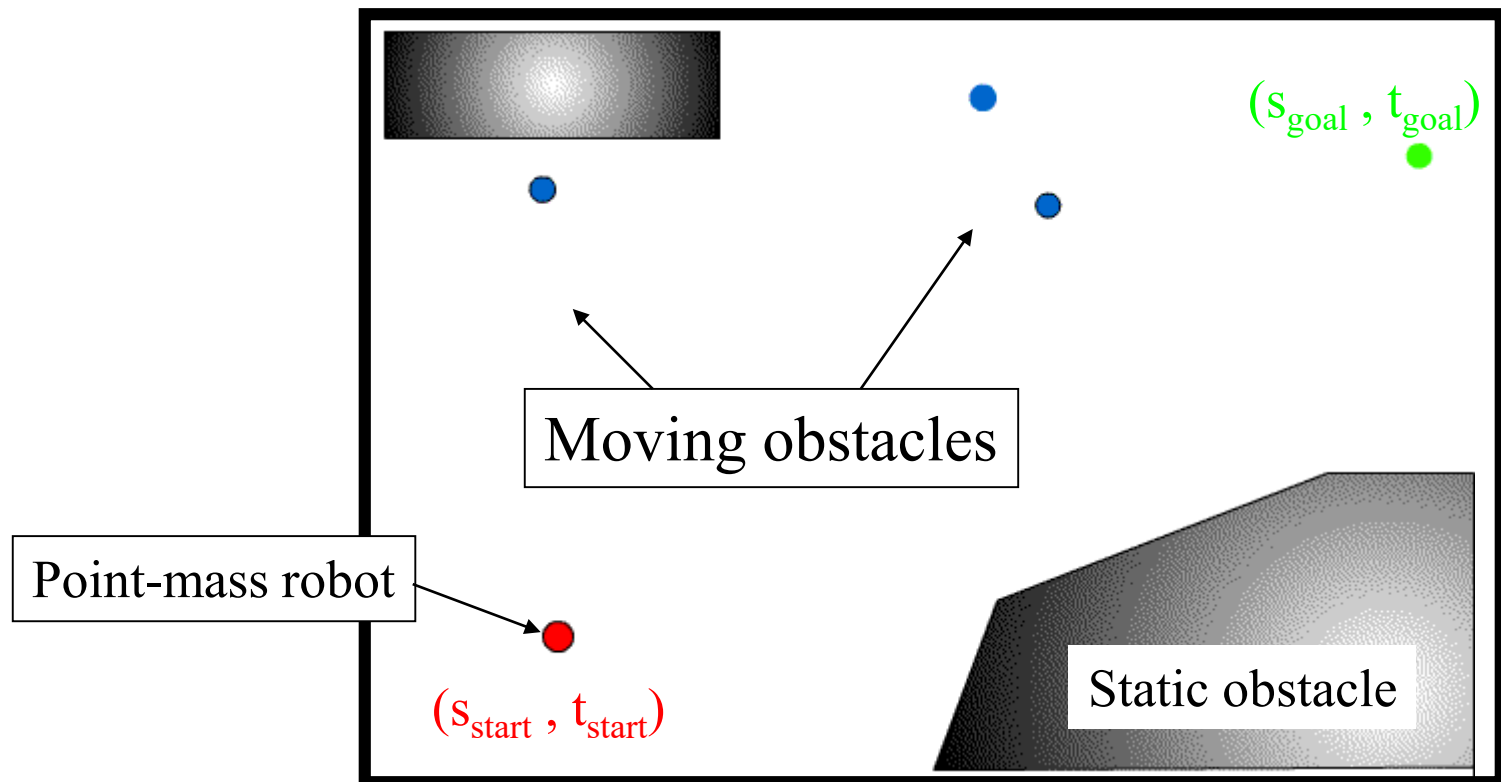
Docking station

MOP Overview

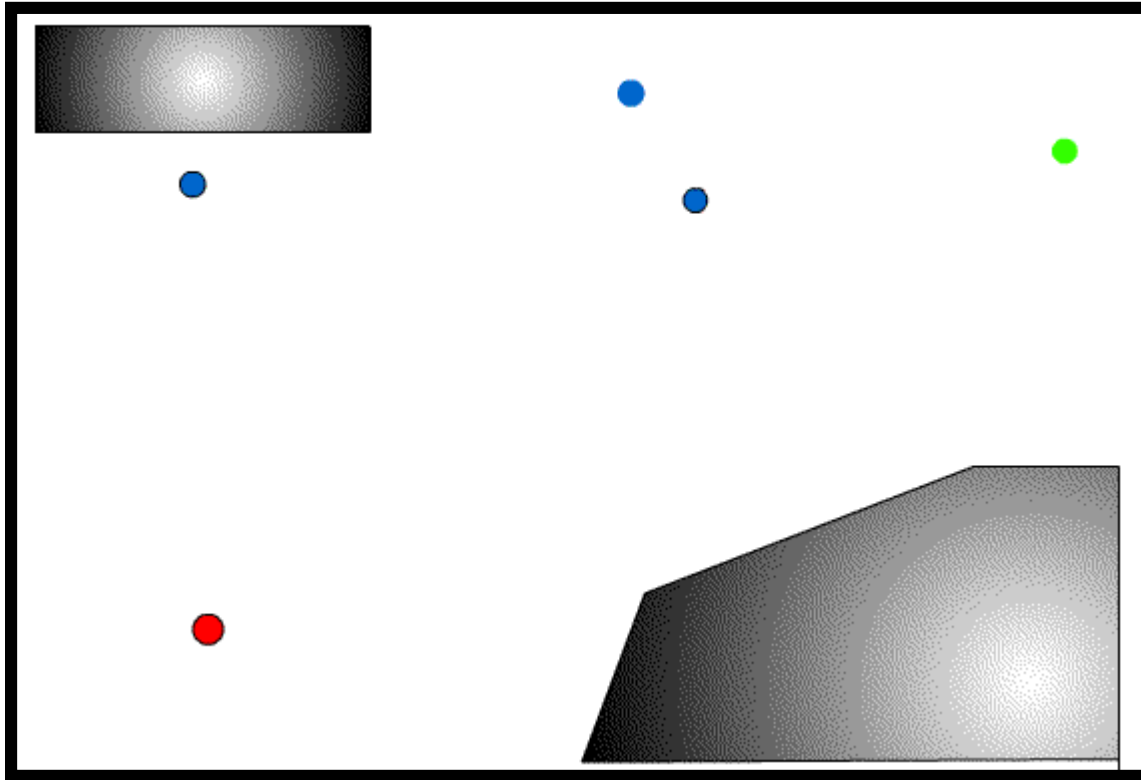
- Similar to PRM, except
- Does not pre-compute the roadmap
- Incrementally constructs the roadmap by extending it from existing nodes (similar to RRT)
- Roadmap is a **directed tree** rooted at initial **state $\times$ time** point and oriented along time axis
- For each query, the planner incrementally builds new roadmap in **state $\times$ time** space
 - **Why?** The environment includes moving obstacles that change location (state) continuously with time
 - Each node in the roadmap must be indexed by both its state and the time it is attained
 - Example: node n is **valid at time t** , however at time **$t+\delta$ node n collides** with a moving obstacle

Demonstration of MOP

- *Point-mass* robot moving in a plane
- State $s = (x, y, \dot{x}, \dot{y})$



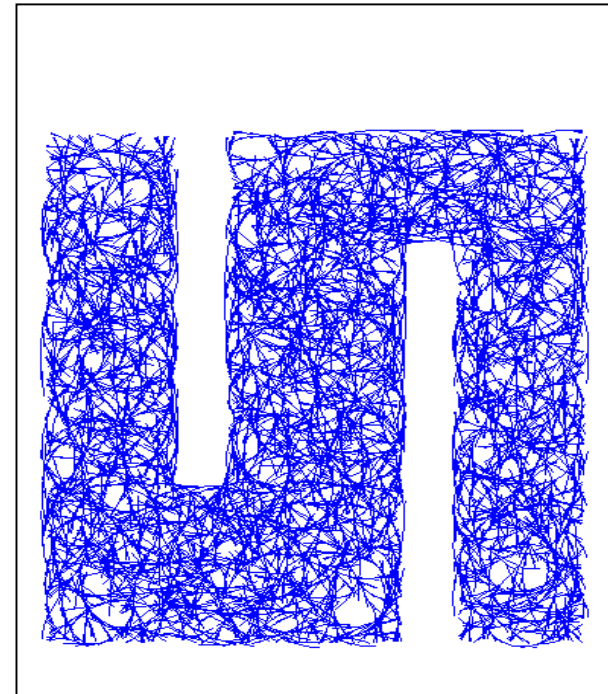
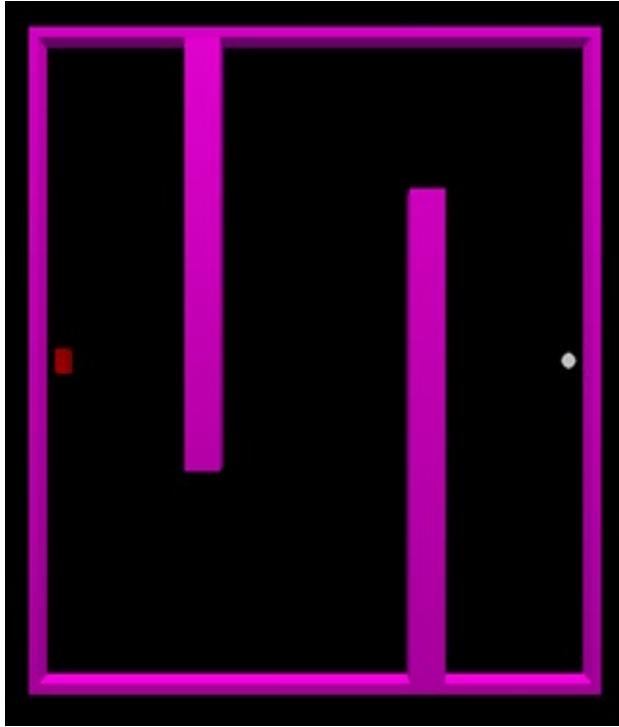
Demonstration of MOP



Planning with RRTs

- RRTs: Rapidly-exploring Random Trees
- Similar to MOP
 - Incrementally builds the roadmap tree
 - Integrates the control inputs to ensure that the kinodynamic constraints are satisfied
- Different exploration strategy from MOP
- Extends to more advanced planning techniques

Example: Simple RRT Planner

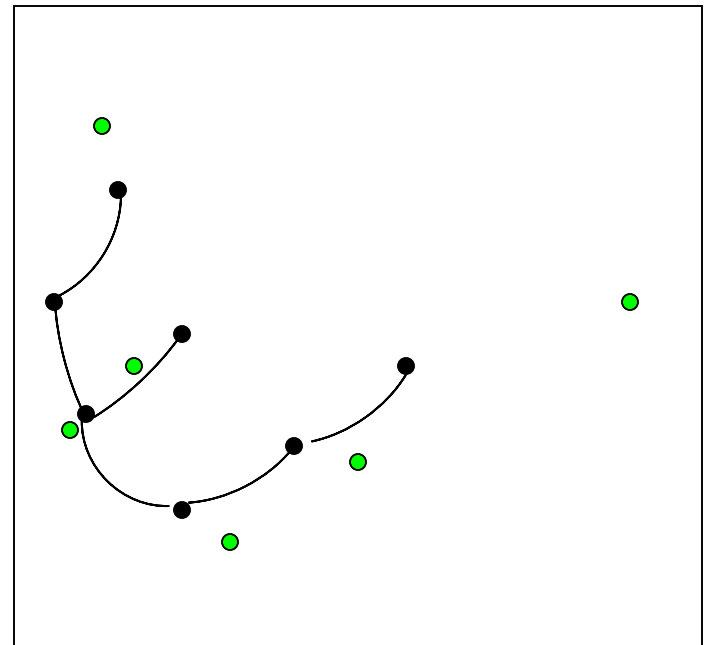
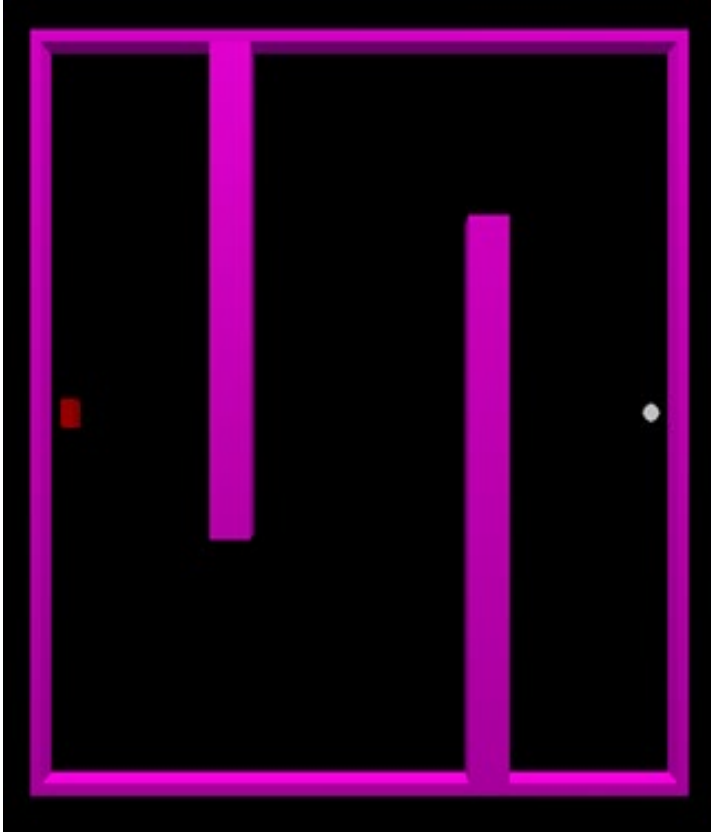


Problem: ordinary RRT explores X uniformly

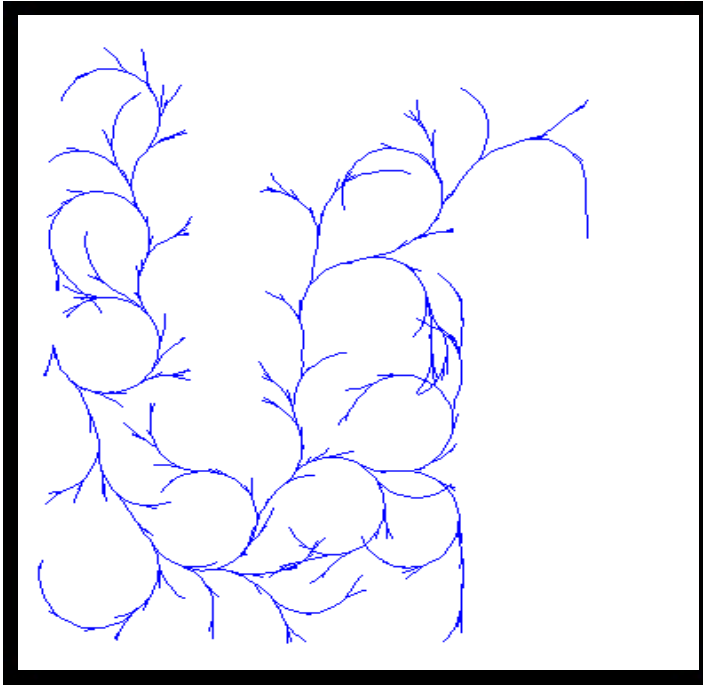
→ slow convergence

Solution: bias distribution towards the goal

Goal-biased RRT



The world is full of local minima

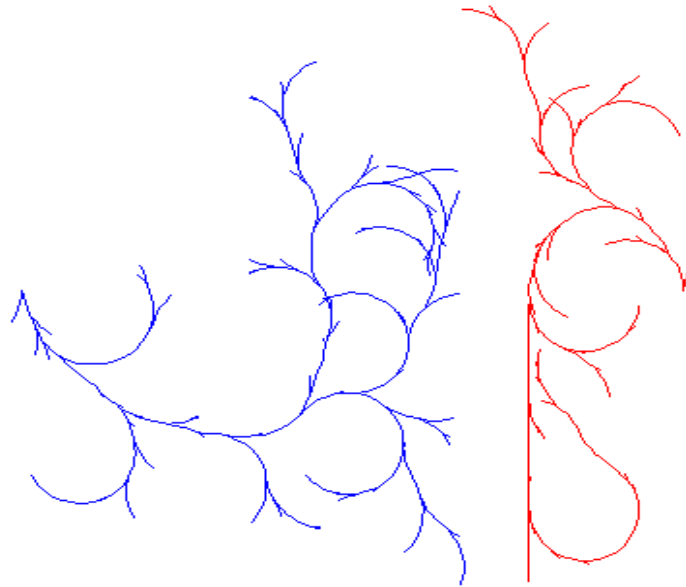


- If too much bias, the planner may get trapped in a local minimum
- A different strategy:
- Pick a point near s_{goal} based on distance from goal to the nearest v in G
- Gradual bias towards s_{goal}

Rather slow convergence

Bidirectional Planners

- Build two RRTs, from start and goal state



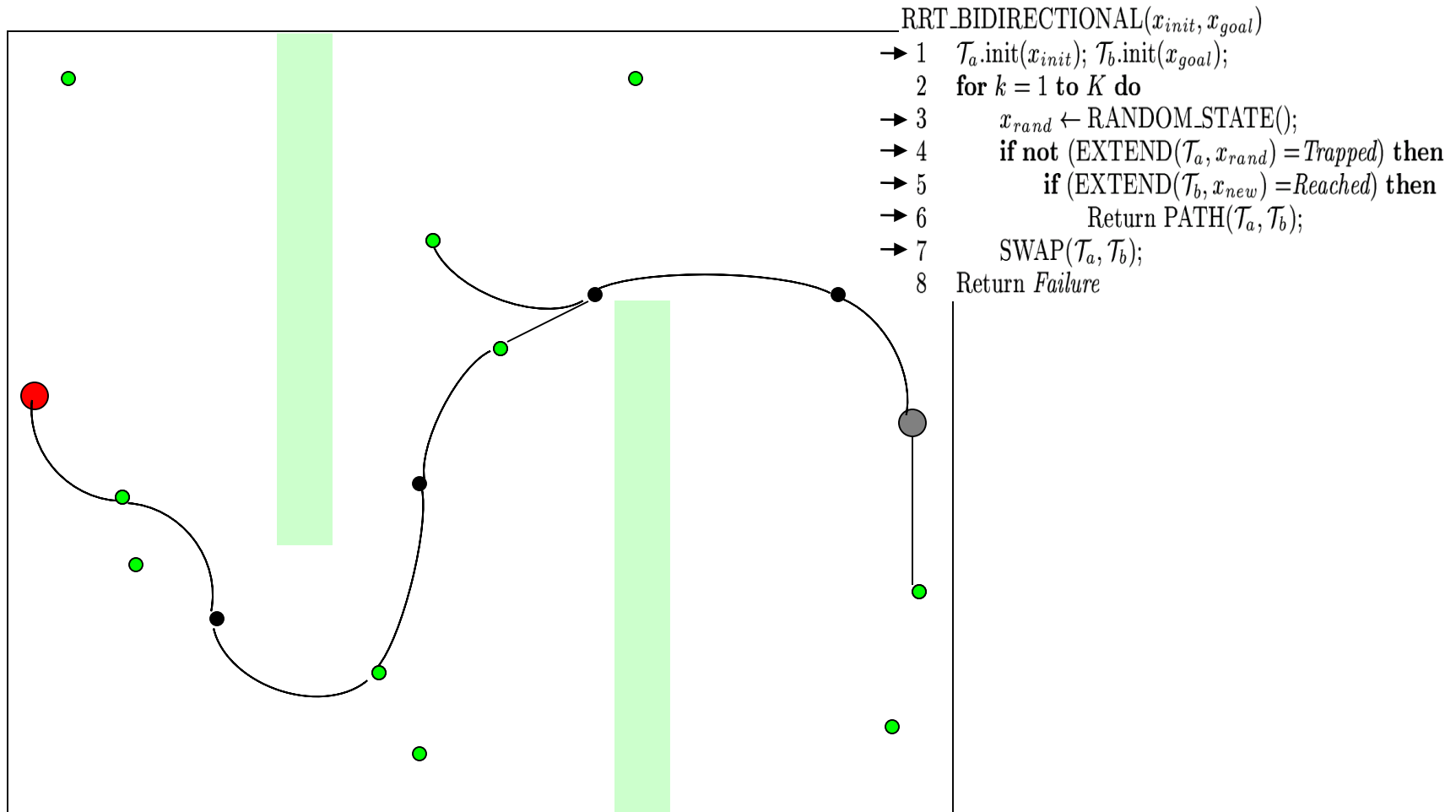
Complication: need to connect two RRTs

- local planner will not work (dynamic constraints)
- bias the distribution, so that the trees meet

Bidirectional Planner Algorithm

```
RRT_BIDIRECTIONAL( $x_{init}, x_{goal}$ )
1   $\mathcal{T}_a.\text{init}(x_{init}); \mathcal{T}_b.\text{init}(x_{goal});$ 
2  for  $k = 1$  to  $K$  do
3       $x_{rand} \leftarrow \text{RANDOM\_STATE}();$ 
4      if not ( $\text{EXTEND}(\mathcal{T}_a, x_{rand}) = \text{Trapped}$ ) then
5          if ( $\text{EXTEND}(\mathcal{T}_b, x_{new}) = \text{Reached}$ ) then
6              Return  $\text{PATH}(\mathcal{T}_a, \mathcal{T}_b);$ 
7           $\text{SWAP}(\mathcal{T}_a, \mathcal{T}_b);$ 
8  Return Failure
```

Bidirectional Planner Example



Bidirectional Planner Example



Conclusions

- Path planners for real-world robots must account for dynamic constraints
- Building the roadmap tree incrementally
 - ensures that the kinodynamic constraints are satisfied
 - avoids the need to reconstruct control inputs from the path
 - allows extensions to moving obstacles problem
- MOP and RRT planners are similar
- Well-suited for single-query problems
- RRTs benefit from the ability to steer a robot toward a point
 - RRTs explore the state more uniformly
 - RRTs can be biased towards a goal or to grow into another RRT